# AI Reactions :

## Korea vs. English Media & Communities

Group 5

2466044 이시은, 2392032 정지영, 2392012 백지현, 2492034 장윤서, 2392047 전서영, 2414022 유선하, 2492045 김채은

Repository: https://github.com/DE-Team5/ai-sentiment-analysis

# INDEX


01 Introduction

02 Project Overview

03 Methodology

04 Implementation

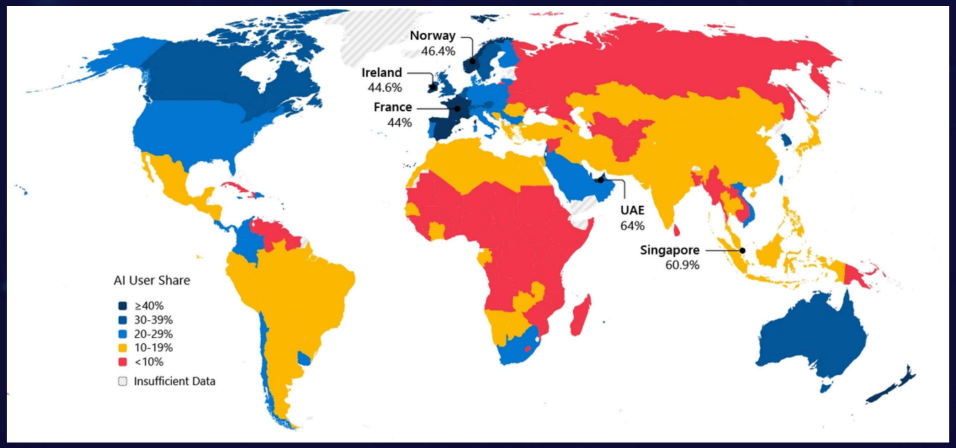
05 Result & Analysis

06 Collaboration

07 Conclusion

# 1. Introduction : Why AI?

# 1. Introduction



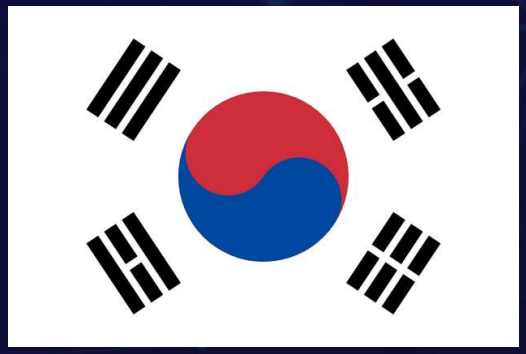

https://www.microsoft.com/en-us/corporate-responsibility/topics/AI-Economy-Institute/reports/Global-AI-Adoption-2025/

- Rapid growth of AI technologies such as ChatGPT
- Increasing influence of AI on media and public opinion
- Potential differences in perception across cultures and languages
- Need to understand how Korean and English-speaking communities respond differently

Gemini Claude

# 1. Introduction

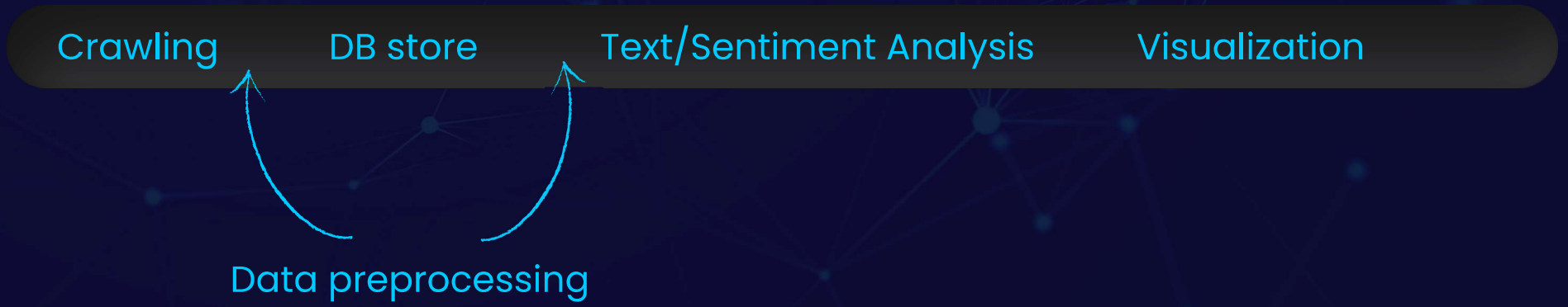
: Focusing on News & Communities



vs

**Kor** **Eng**

# 2. Process Overview



# 3. Methodology
# : Crawling

# 3. Methodology- Crawling

## Target Data Sources

| | Korean source | English source |
|---|---|---|
| **Community** | Ewha Everytime, Blind, DC Inside, Nate Pann, Youtube | Reddit, Youtube |
| **News** | Naver News | The Guardian |

# 3. Methodology- Crawling

## Criteria

- To compare reactions across communities, data was collected under controlled conditions.
- Keywords: 인공지능, Artificial Intelligence, 에이아이, AI, 지피티, GPT
- Community
  - Sort by posted time (newest)
  - Crawl 300 posts with comments for each keyword
- News
  - Define 4 time periods ("2021.10.30-2022.11.29", "2022.11.30-2023.12.29","2023.12.30-2025.01.28","2025.01.29-2026.03.30") in accordance with changes in the AI paradigm.
  - Sort by relevance

# 3. Methodology- Crawling

## Basic data schema - json format


```
{
  "id": "1sn2rp0",
  "subreddit": "Solopreneur",
  "keyword": "AI",
  "keywords_matched": "AI",
  "title": "Almost by accident I've created a tool for solo founders to build an organic TikTok growth channel for their product. Looking for your opinion",
  "body": "I started this year from building an iOS app for dancers. And needless to say, I launched to crickets - all downloads only from friends)\n\nConcurrent
  "author": "vkjr",
  "score": 1,
  "num_comments": 0,
  "created_date": "2026-04-16",
  "month": "2026-04",
  "permalink": "https://www.reddit.com/r/Solopreneur/comments/1sn2rp0/almost_by_accident_ive_created_a_tool_for_solo/",
  "comments": [
    {
      "comment_id": "ogihw15",
      "author": "tanishkacantcopee",
      "body": "30–50 downloads/day from organic TikTok is meaningful if those users retain",
      "score": 1,
      "created_at": "2026-04-16T12:58:09",
      "depth": 0
    },
    {
      "comment_id": "ogii7b3",
      "author": "vkjr",
      "body": "Yeah, it is not bad. Of course user retention is more about your product/onboarding and about \"match\" of the audience that you brought to the au
      "score": 1,
      "created_at": "2026-04-16T12:59:47",
      "depth": 1
    }
  ]
},
```


Original post information

- keyword
- title
- content
- posting time

Comments

- content
- posting time

# 3. Methodology- Crawling

- requests + BeautifulSoup (Nate Pann)


- **Approach (Design) :**
  - Efficient parsing of static HTML content
  - Fast and lightweight compared to browser automation tools
- **Nate Pann**
  - Mostly server-rendered HTML
  - Main content are available directly in the page source

# 3. Methodology- Crawling

- Playwright (Naver News + Reddit)


- **Approach (Design) :**
- Launches headless Chromium to fully render JavaScript
- Naver: 2-stage router
  - SEARCH handler collects URLs
  - ARTICLE handler extracts title, body, etc.
- Reddit: infinite scroll + .json endpoint for comments

- **Rationale (Selection Reason) :**
- Naver search results are JS-rendered- static HTTP fails
- Reddit official API: OAuth approval was blocked
- Third- party Reddit APIs lack recent data
- Playwright mimics real browser → bypasses bot detection
- Crawlee adds concurrency, retry logic, request queue

# 3. Methodology- Crawling

- Playwright (Everytime+Blind+dc inside)


- **Everytime**
  - Requires user login
  - Content loaded dynamically via infinite scroll, ‘Next’ button
- **Blind**
  - Feed is built on infinite scroll structure
  - Content is heavily dependent on JavaScript rendering
- **DC inside**
  - Ensures stable page rendering
  - Moderate anti-bot protection

# 3. Methodology- Crawling

- The Guardian - Guardian API

The Guardian

- **Approach (Design) :**
- Sends HTTP GET requests with keyword and date-range params
- Retrieves bodyText, headline, byline directly in response
- Sorted by relevance to match Naver's collection approach

- **Rationale (Selection Reason) :**
- Official API provides structured JSON - no HTML parsing needed
- Legally compliant and stable (no bot detection risk)
- Supports data-range filtering to retrieve period-specific data

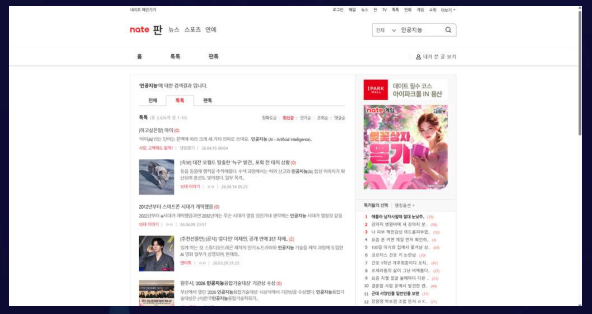
# 3. Methodology- Crawling

## Choosing crawling tools - Summary

- requests + BeautifulSoup
  - Used for websites with static HTML content
  - Suitable site: Nate Pann
- Playwright
  - Used for websites with dynamic content rendered through JavaScript
  - Suitable site: Everytime, Blind, DC Inside, Naver News, Reddit
- API
  - Used when the platform provides an official or unofficial endpoint that returns data
  - Suitable site: The Guardian, Youtube

# 3. Methodology- Crawling

- Nate pann
  - Search results were accessible through static HTTP requests
  - Latest sorting was controlled through URL parameters (sort=DD)
  - The search page is accessed directly through a URL with query parameters for the keyword and page number.
  - Crawling flow:
    - Keyword search → collect post IDs from search pages → open post detail page → extract post content → request comment API pages → request nested reply API → save as JSON



Search page with '인공지능' keyword

pann.nate.com/search/talk?q=인공지능&page=2

pagination through structured URL

# 3. Methodology- Crawling

- Nate pann
  - Technical challenges address
    - Comments and nested replies were fetched through separate API requests

```python
def fetch_reply_page(session: requests.Session, pann_id: str, page: int, order: str = "W") -> BeautifulSoup:
    params = {
        "pann_id": pann_id,
        "reply_id": "0",
        "rereply_id": "0",
        "page": str(page),
        "penm": "",
        "order": order,
    }
    r = session.get(f"{NATE_BASE}/talk/reply/load", params=params, timeout=30)
    r.raise_for_status()
    return BeautifulSoup(r.text, "lxml")
```

# 3. Methodology- Crawling

- Naver news
  - Search pages were queried per keyword x date range with structured pagination URLs
  - Article detail pages (external press) were parsed from server-rendered HTML using multi-selector fallback
  - Collected up to 300 articles per keyword("AI","인공지능", "GPT") × period combination (3 keywords × 4 periods)

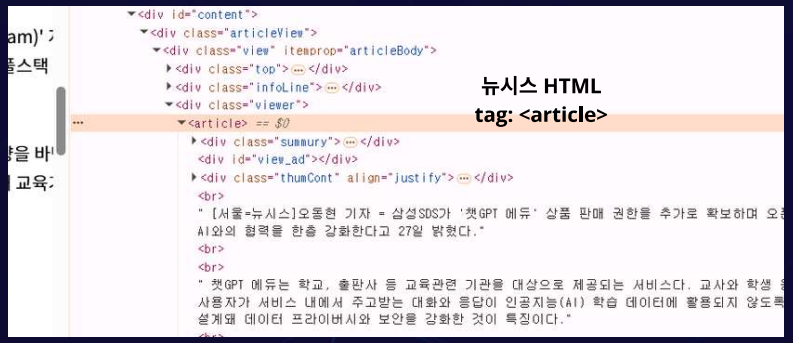
  - Crawling flow:
    - Keyword x period search → iterate paginated search pages
      → collect article URLs → open each article page
      → wait for content to load
      → extract data from article
        : title, body, author, created_at, press, category, period etc.
      → filter excluded keywords →  save as JSON


```
{
  "keyword": [
    "AI"
  ],
  "source": "naver_news",
  "source_type": "news",
  "url": "https://n.news.naver.com/mnews/article/022/0004116888",
  "post_id": "0004116888",
  "title": "서울대, AI 경쟁력 강화…연봉 3억원 석학 5명 영입 추진",
  "content": "서울대가 인공지능(AI) 분야 연구 역량을 강화하고 인재를 양성하기 위해 석학
  "author": "구예지 기자",
  "created_at": "2026-03-30 22:02:27",
  "period": "current",
  "board": "사회",
  "like_count": null,
  "scrap_count": null,
  "comment_count": 0,
  "press": "세계일보",
  "language": "kr",
  "category": "사회"
},
```

# 3. Methodology- Crawling

- Naver news - Technical Challenges Addressed

## Multi-Selector Fallback


Problem : Structural inconsistency across News domains
- Single- selector crawling leads to high failure rates and data gaps.

Solution : 4- Tier Cascading fallback logic

- Step 1: High- Precision semantic marker( [itemprop=’articleBody’])
- Step 2: Heuristic pattern matching
  - Applying common ID/Class conventions used across major outlets(e.g., #articleBody, .article_body, etc.)
- Step 3: Structural tag extraction
  - Capturing the entire <article> element as a broader container fallback
- Step 4: Element aggregation & reconstruction
  - Last-resort extraction and merging of all <p> tags to reconstruct the body text

# 3. Methodology- Crawling

- Naver news - Technical Challenges Addressed

  Multi-Selector Fallback

```python
async def main():
    async def handle_article(context: PlaywrightCrawlingContext):
        # 본문
        content = ""
        for sel in [
            "[itemprop='articleBody']", "#articleBody", "#article-view-content-div",
            "#cont_newstext", "div#articletxt",
            ".article_body", ".article-body", "#article_body",
            ".news_body", ".news_content", "#news_body_area",
            ".view_cont", ".view_con", ".view_article",
            "#textBody", "#news_detail", ".detail_body",
            ".article_txt", "#article_text", ".news_view",
            "article",
        ]:
            el = await page.query_selector(sel)
            if el:
                content = (await el.inner_text()).strip()
                if len(content) > 100:
                    break

        # fallback 1: p 태그 모아서 본문 구성
        if not content or len(content) < 100:
            paragraphs = await page.query_selector_all("p")
            texts = []
            for p in paragraphs:
                t = (await p.inner_text()).strip()
                if len(t) > 30:
                    texts.append(t)
            content = "\n".join(texts)
```

Step 1: [itemprop='articleBody'] -schema.org standard

Step 2: Other tags except [itemprop='articleBody'] and article - commo patterns from press sites

Step 3 :Extract tag <article>  (like example from 뉴시스)

Step 4: <p> tag aggregation

# 3. Methodology- Crawling

- Reddit
  - Use Playwright browser automation library to launch a real browser and crawl the Reddit search page directly.
  - Two-stage architecture: first, it collects a list of posts via infinite scrolling, and then it fetches the full content and comments separately using Reddit's .json endpoints.
  - Collected up to 300 posts per keyword ("AI","Artificial Intelligence", "gpt")

- Crawling flow:
  Keyword search (sort=new) → launch Playwright Chromium browse
  → load search page ( scroll& wait for load)
  → Extract posts from HTML(just posts):
  id, title, subreddit, author,num_comments, permalink etc.
  → open each post's .json endpoint
  → Extract post body, comments and replies:
  comment_id, author, body, created_at -> save as JSON


```
{
  "id":"1nfcq6a",
  "subreddit":"law",
  "keyword":"gpt",
  "keywords_matched":"gpt",
  "age_group":null,
  "job_group":"Law",
  "title":"US DOJ blocks access to their own National Institute of Justice research artic
  "body":"Edit: can't post images so here is a link to my post that hopefully won't get d
  "url":"https:\/\/nij.ojp.gov\/topics\/articles\/what-nij-research-tells-us-about-domest
  "author":"PatientExcuse6273",
  "score":48174,
  "upvote_ratio":0.9700000286,
  "num_comments":711,
  "created_utc":1757706083,
  "created_date":"2025-09-12",
  "month":"2025-09",
  "permalink":"https:\/\/reddit.com\/r\/law\/comments\/1nfcq6a\/us_doj_blocks_access_to_t
  "flair":"Other",
  "domain":"nij.ojp.gov",
  "kw_in_title":false,
  "kw_in_body":true,
  "comments":"[{\"comment_id\": \"net4ycv\", \"content\": \"It's here:  [https:\/\/limewi
  "group":"Law",
  "primary_keyword":"GPT"
},
```

# 3. Methodology- Crawling

- Reddit - Technical Challenges Addressed

  Handling Rate Limit(HTTP 429)

```python
# ── JSON fetch (댓글/본문) ──────────────────────────────────────

async def fetch_json(page: Page, url: str, retries: int = 5):
    for attempt in range(1, retries + 1):
        try:
            resp = await page.goto(url, wait_until="domcontentloaded", timeout=30_000)
            if resp and resp.status == 429:
                wait = 30 * attempt
                print(f"  ⚠️  429 – {wait}초 대기 ({attempt}/{retries})")
                await asyncio.sleep(wait)
                continue
            if resp and resp.status >= 500:
                await asyncio.sleep(10 * attempt)
                continue
            text = await page.inner_text("pre")
            return json.loads(text)
        except Exception:
            await asyncio.sleep(10 * attempt)
    return None
```

Problem- “HTTP 429 Too Many Requests” error:

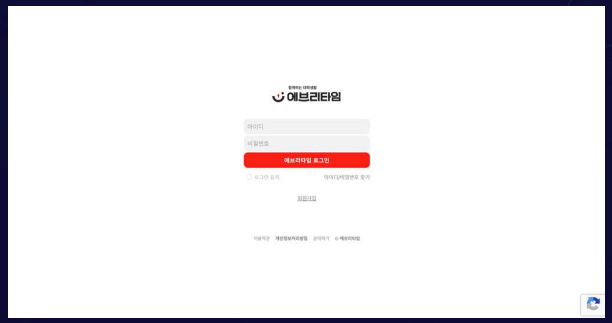
- When a high volume of requests is detected in a short period.
- The request frequency can easily trigger these limits since the script requests the .json endpoint for every individual post (body and comments)

Solution-Implement an exponential backoff and retry logic:

- fetch_json(): if 429 response is received, the script waits for seconds before trying

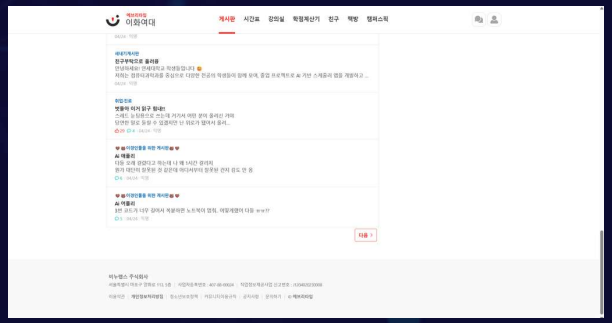
# 3. Methodology- Crawling

- Everytime
  - Requires user login. Save ID, password info in a .env file
  - Content is sorted by recent post, and  rendered with JS
  - Need to push ‘next’ button to see the next page




Login page


Search page with ‘AI’ keyword




HTML based pagination with <a> tag

# 3. Methodology- Crawling

- Everytime
  - Crawling flow:
    - Login → Search keyword → Collect post URLs → Navigate pages (Numeric pages) → Visit each post → Extract metadata + comments → Save as JSON
  - Technical challenges addressed
    - Login Automation: Implemented code for automated login. The user ID and password are stored in a .env file for security purposes

```python
def login(page, user_id, user_pw):
    page.goto(LOGIN_URL)

    fill_first_available(page, ID_SELECTORS, user_id)
    fill_first_available(page, PW_SELECTORS, user_pw)

    if click_if_exists(page, SUBMIT_SELECTORS):
        pass
    else:
        page.keyboard.press("Enter")

    page.wait_for_load_state("networkidle")
```

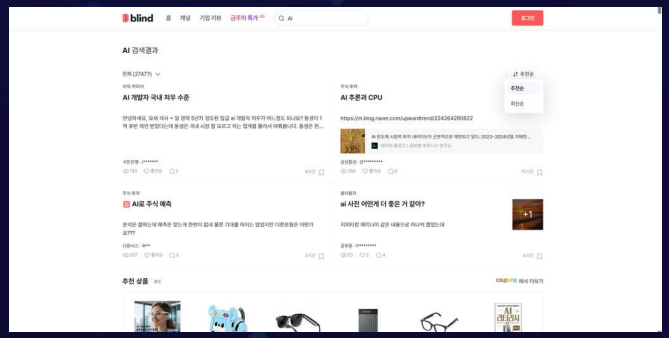
Login Automation logic

```
.env
1  EVERYTIME_ID=(***user id***)
2  EVERYTIME_PW=(***user password***)
```

Saved ID, password in .env file

# 3. Methodology- Crawling

- Blind
  - Search results were rendered dynamically through JavaScript
  - Latest sorting was controlled through UI interactions rather than URL parameters
  - Search results were loaded through an infinite scroll structure
  - Crawling flow:
    - Keyword search → apply latest sort → auto-scroll to load posts → collect post URLs → extract post details → expand comments → save as JSON




Search page with ‘AI’ keyword

# 3. Methodology- Crawling

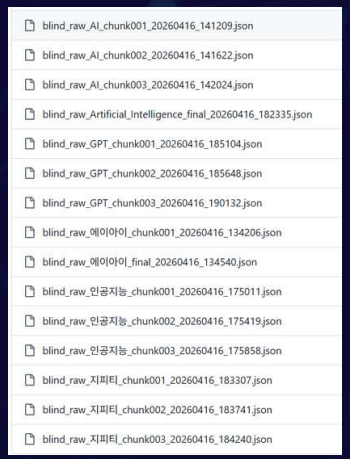
- Blind
  - Technical challenges addressed
    - Data Loss Prevention During Long Collection Times: Due to long scraping times caused by heavy JavaScript rendering, collected data could be lost when errors occurred. To address this, logic was added to save the file in batches of every 100 posts so that data collected up to that point would be preserved.

```python
def print_progress(keyword: str, count: int, started_at: float) -> None:
    if not should_log_progress(count):
        return
    elapsed = time.perf_counter() - started_at
    print(f"[{keyword}] Collected {count}/{MAX_POSTS_PER_KEYWORD} posts (elapsed: {elapsed:.1f}s)", flush=True)
```

Print the crawling progress by collected posts

```python
def save_chunk(keyword: str, records: list[dict[str, Any]], chunk_index: int, final: bool) -> Path:
    out_dir = ensure_output_dir()
    ts = datetime.now().strftime("%Y%m%d_%H%M%S")
    suffix = "final" if final else f"chunk{chunk_index:03d}"
    out_file = out_dir / f"blind_raw_{_safe_name(keyword)}_{suffix}_{ts}.json"
    payload = {
        "crawl_id": f"blind_{_safe_name(keyword)}_{ts}",
        "source": "blind",
        "keyword": keyword,
        "fetched_at": utc_now_iso(),
        "records": records,
    }
    out_file.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
    return out_file
```

save .json by chunk of 100 posts




Saved data

# 3. Methodology- Crawling

- Blind
  - Technical challenges addressed
    - Automated Latest-First Sorting: Implemented DOM-based interaction logic to automatically locate the sorting menu button (<a class="state">) and select the “Latest” option from the popup menu ( … <a>최신순</a>). This allows the crawler to switch search results from the default relevance order to chronological order before scraping begins, ensuring recent posts are collected first.

```
try:
    desktop_state = page.locator(".wrap-category-pc .sort a.state").first
    if desktop_state.count() > 0:
        desktop_state.scroll_into_view_if_needed(timeout=3000)
        desktop_state.click(timeout=4000, force=True)
        page.wait_for_timeout(300)
        latest_opt = page.locator('#search_sort a:has-text("최신순")').first
        if latest_opt.count() > 0:
            latest_opt.click(timeout=4000, force=True)
            page.wait_for_timeout(1200)
            if is_latest_sort_applied(page):
                print("[sort] Latest sort selected via #search_sort.", flush=True)
                return True
except Exception:
    pass
```

latest-sort implementation

# 3. Methodology- Crawling

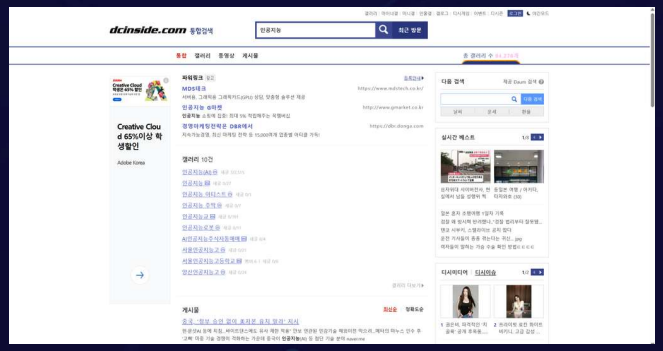
- Blind
  - Technical challenges addressed
    - Infinite Scroll Handling: Implemented automated scrolling logic using browser-side JavaScript (window.scrollTo(0, document.body.scrollHeight)) to continuously load additional posts on dynamically rendered search pages. The crawler repeatedly scrolls and monitors newly loaded content until the target number of posts is collected or no further results appear.

```
# Scroll then pause briefly to trigger additional load.
try:
    page.evaluate("window.scrollTo(0, document.body.scrollHeight)")
except Exception:
    pass
page.wait_for_timeout(700)
page.wait_for_timeout(450)
```

scroll logic

# 3. Methodology- Crawling

- DC inside
  - Search pages were accessed through structured pagination URLs
  - Post detail pages and comments were parsed from server-rendered HTML
  - Crawling flow:
    - Keyword search → move through paginated search pages → collect candidate post URLs → open each post page → wait for comments to load → extract post details and comments → save as JSON




Search page with '인공지능' keyword

```
# Iterate through paginated search pages and extract post listings
page.goto(search_url(keyword, page_num))
hits = parse_search_hits(page.content(), keyword)

# Open each collected post URL and wait for comments to load
page.goto(post_url)
wait_comments_loaded(page)

# Extract post page HTML data
row = parse_post_page(page.content(), ...)
save_json(records)
```

# 3. Methodology- Crawling

- Youtube
  - This project implements API-driven data collection using the YouTube Data API v3.
  - It interfaces with official Google endpoints to retrieve video metadata, metrics, and comments.
  - Collected up to 300 videos per keyword (ko:"ai","인공지능", "gpt", en: "ai","artificial intelligence", "gpt")

  - Crawling flow:
    - Keyword x language(ko, en) search

→ Retrieve a list of video_ids matching the keywords

→ pass IDs to videos.list api to extract data about the vide
: title, description, channel, category, likes etc.

→ Collect top-level comments and corresponding replies

→ save as JSON


```
{
  "video_id": "X0bGTdKspt0",
  "url": "https://www.youtube.com/watch?v=X0bGTdKspt0",
  "title": "ARTIFICIAL INTELLIGENCE AND APPLIED INNOVATION",
  "channel": "La Plage Services",
  "category": "Entertainment",
  "published_at": "2026-04-16T18:48:12Z",
  "view_count": 134,
  "like_count": 16,
  "comment_count": 3,
  "audio_language": "en",
  "comments": [
    {
      "author": "@CreativeBrainsInternational",
      "text": "Learning and growing with La Plage Meta is top notch. Thank you  great team for
      "likes": 0
    },
    {
      "author": "@justinaodigie-h9q",
      "text": "The sessions were highly insightful, illustrative, informative and engaging.Than
      "likes": 0
    },
    {
      "author": "@ParadiseisReal-p5u",
      "text": "Thank you.",
      "likes": 1
    }
  ],
```

# 3. Methodology- Crawling

- Youtube - Technical Challenges Addressed
  1. API Quota limit

```python
# ── Checkpoint 관리 ──────────────────────────
def load_checkpoint():
    if os.path.exists(CHECKPOINT_FILE):
        with open(CHECKPOINT_FILE, "r", encoding="utf-8") as f:
            print(f"[체크포인트] 이전 진행 상황을 불러왔습니다.")
            return json.load(f)

    return {
        "last_updated":    None,
        "collected_ids":   [],
        "search_progress": [
            {
                "keyword":           c["keyword"],
                "relevanceLanguage": c["relevanceLanguage"],
                "nextPageToken":     None,
                "collected_count":   0,
                "done":              False
            }
            for c in SEARCH_CONFIGS
        ]
    }

def save_checkpoint(checkpoint):
    checkpoint["last_updated"] = datetime.now().isoformat()
    with open(CHECKPOINT_FILE, "w", encoding="utf-8") as f:
        json.dump(checkpoint, f, ensure_ascii=False, indent=2)
```

Problem - Daily API Quota limits:

- Impossible to collect all the data in one day
- Previously collected videos reappear when we just restart it tomorrow and spend meaningless quota

Solution - Implement a checkpoint system:

- resume exactly from where it left off when the script is rerun

# 3. Methodology- Crawling

- Youtube - Technical Challenges Addressed

  2. Incomplete Retrieval of Nested Replies

```python
def get_comments(video_id, max_results=20):
        # 대댓글 처리
        # replies 필드에 일부 포함되지만 전체가 아닐 수 있으므로
        # replyCount > 0 이면 comments.list로 전체 대댓글 별도 수집
        reply_count = item["snippet"].get("totalReplyCount", 0)
        if reply_count > 0:
            replies_in_thread = item.get("replies", {}).get("comments", [])

            # API가 대댓글을 전부 반환했는지 확인
            if len(replies_in_thread) < reply_count:
                # 전부 없으면 comments.list로 추가 수집
                comment["replies"] = get_replies(thread, max_results=100)
            else:
                # 이미 전부 포함된 경우
                comment["replies"] = []
                for r in replies_in_thread:
                    s   = r["snippet"]
                    raw = {
                        "author": s.get("authorDisplayName"),
                        "text":   s.get("textDisplay"),
                        "likes":  s.get("likeCount")
                    }
                    cleaned = remove_empty_fields(raw)
                    if cleaned:
                        comment["replies"].append(cleaned)

            if not comment["replies"]:
                comment.pop("replies", None)
```

Problem - commentThreads.list API often returns only a partial subset of nested replies:

- The number of replies actually received are less than it should be → data loss

Solution - Implement a validation step:

- Compares the number of retrieved replies against totalReplyCount from commentThreads.list API
- Trigger supplementary call if there is missing replies

# 3. Methodology- Crawling

- Youtube - Technical Challenges Addressed
  3. Mixed Language Results

```python
# ── 기존 결과에 language 태그 추가 ──────────────
def patch_language():
    for path in [RESULT_FILE]:
        if not os.path.exists(path):
            print(f"  파일 없음: {path}")
            continue
        with open(path, "r", encoding="utf-8") as f:
            data = json.load(f)

        changed = 0
        for d in data:
            al = d.get("audio_language", "")
            cl = d.get("channel_language", "")
            title = d.get("title", "")
            if al.startswith("ko") or cl.startswith("ko") or has_korean(title):
                d["language"] = "ko"
            elif al.startswith("en") or cl.startswith("en") or has_english(title):
                d["language"] = "en"
            else:
                d["language"] = "other"
            changed += 1

        with open(path, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
        print(f"  {path}: {changed}개 태그 추가 (총 {len(data)}개)")
```

Problem - API's relevanceLanguage parameter does not guarantee language-specific results:

- Initial dataset contained mixed- language entries
-

Solution - Add filter logic has_korean(), has_english():

- Check audio/channel metadata and regex- based on Unicode detection on titles
- Allowed for precise language tagging and the removal of irrelevant videos that failed to match the target keyword language.

# 3. Methodology- Crawling

- Guardian News
  - Rather than scraping HTML directly, the official REST API endpoint (content.guardianapis.com/search) was queried.
  - Collected up to 300 articles per keyword("AI","artificial intelligence", "GPT") × period combination (3 keywords × 4 periods)

  - Crawling flow:
    - Keyword x period search
    - → extract data directly from the API responses:
      : title, content, created_at, keyword, period, url, etc.
    → save as JSON


```
{
  "keyword": [
    "AI",
    "artificial intelligence"
  ],
  "source": "guardian",
  "source_type": "news",
  "url": "https://www.theguardian.com/technology/2022/oct/14/ai-da-robot-sums-up-
  "post_id": "ai-da-robot-sums-up-flawed-logic-lords-debate-ai",
  "title": "Ai-Da the robot sums up the flawed logic of Lords debate on AI",
  "content": "When it announced that “the world’s first robot artist” would be gi
  "author": "Alex Hern",
  "created_at": "2022-10-14 16:05:33",
  "period": "2021-10-30~2022-11-29 (before)",
  "board": "technology",
  "like_count": null,
  "scrap_count": null,
  "comment_count": null,
  "press": "The Guardian",
  "language": "uk",
  "category": "technology"
},
```

# 3. Methodology- Crawling

- Guardian API - Technical Challenges Addressed

```
ef load_checkpoint() -> dict:
    체크포인트 구조:
    {
        "period_index":    0,   # DATE_RANGES 인덱스
        "keyword_index":   0,   # KEYWORDS 인덱스
        "page":            1,   # 현재 페이지
        "collected_today": 0,   # 오늘 사용한 요청 수
        "last_reset_date": "YYYY-MM-DD"
    }
    """
    path = Path(CHECKPOINT_FILE)
    if path.exists():
        with open(path, "r", encoding="utf-8") as f:
            cp = json.load(f)
        logger.info(
            f"체크포인트 로드 – "
            f"period_idx={cp['period_index']}, "
            f"keyword_idx={cp['keyword_index']}, "
            f"page={cp['page']}, "
            f"오늘 요청={cp['collected_today']}건"
        )
        return cp
    return {
        "period_index":    0,
        "keyword_index":   0,
        "page":            1,
        "collected_today": 0,
        "last_reset_date": _today(),
```

Problem- Daily API rate limit management:

- The Guardian free API enforces a daily request cap (~500 calls/day)
- 300 articles x 3 keywords x 4 periods requires hundreds of paginated calls,
  a single run cannot complete in one day.

Solution:

- Add guardian_checkpoint file to save where the crawler stopped.
- On re-run load_checkpoint() resumes from the exact triple instead of restarting from scratch.

# 3. Methodology- Crawling

## Crawling results

- Collected 19506 posts in total
- Saved as JSON for preprocessing prior to database storage

# 3. Methodology- Crawling

## Additional data collection

- After comparing reactions across communities, additional data collection and analysis were conducted to examine responses to AI by **occupation and interest group**.

- The target platform was **Reddit**, and subreddits matching the desired occupational categories were selected to collect keyword-based posts and comments.
- To select subreddits, those with relatively high weekly visitor counts were chosen to ensure active communities.

- Keywords: Artificial Intelligence, AI, GPT, claude, gemini

| Group | Related Fields | Subreddits |
|---|---|---|
| IT | Programming, Data Science | programmer, programming, softwaredevelopment, datascience, DataScientist, AskEngineers, software, webdev, gamedev |
| Engineers | Civil, Mechanical, Electrical, Food, Environmental | Construction, Contractor, MechanicalEngineering, electrician, ElectricalEngineering, foodscience, environment, Enviornmentalism |
| Natural Science | Math, Biology, Physics, Chemistry, Statistics | askmath, math, biology, AskPhysics, Physics, EverythingScience, chemistry, statistics |
| Teachers | Education, Teaching | Teachers, AskTeachers, TeacherReality |
| Artists | Music, Art, Dance, Design | musicians, Artists, Dance, Design, graphic_design |
| Liberal Arts | Philosophy, History, Literature, Language, Religion | askphilosophy, philosophy, history, AskHistory, literature, writers, translator, AskAPriest |
| Social Sciences / Business | Psychology, Sociology, Politics, Economics, Public Service | Psychologists, askapsychologist, sociology, AskSociology, diplomats, TheDiplomat, theeconomist, librarians, CanadaPublicServants, TheCivilService |
| Medicine | Medicine, Nursing, Pharmacy, Public Health | doctorsUK, DoctorWhumour, Nurses, Nurse, StudentNurse, nursepractitioner, newgradnurse, pharmacy, PharmacyTechnician, publichealth |
| Law | Law, Legal Education | LawSchool, Lawyertalk, law, AceAttorney |

# 3. Methodology- Crawling

- **Reddit**
  - The previous version crawled Reddit directly,But here we use the Arctic Shift API
  - Collect a larger volume of data from specific professional subreddits
  - Arctic Shift: third-party service that archives Reddit data
  - Collected without limits per keyword("AI", "Artificial intelligence", "gpt", "claude", "gemini")

  - Crawling flow (Groups x Subreddits x 5 Keyword):
    Call Arctic Shift API (POSTS_SEARCH endpoint)
    with subreddit and keyword
    → cursor-based pagination
    → filter by duplicate id & keyword substring match
    → fetch comments per post → save as JSON

# 3. Methodology- Crawling

## **Reddit -** Technical Challenges Addressed

Full data retrieval via Cursor-based pagination

```python
def fetch_posts(subreddit: str, keyword: str) -> list[dict]:
    params = {
        "subreddit": subreddit,
        "query": keyword,
        "limit": PAGE_LIMIT,
        "sort": "desc",
    }
    if before is not None:
        params["before"] = before  ← Cursor

    resp = get_with_retry(POSTS_SEARCH, params)
    if resp is None:
        break

    try:
        payload = resp.json()
    except json.JSONDecodeError:
        print("  ❌ JSON 파싱 실패")
        break

    items = payload.get("data", payload) if isinstance(payload, dict) else payload
    if not isinstance(items, list) or not items:
        print("  → 더 이상 게시글이 없습니다.")
        break

    oldest_ts = None
    for p in items:
        ts = p.get("created_utc")
        if ts is not None:
            oldest_ts = ts if oldest_ts is None else min(oldest_ts, ts)
        collected.append(p)

    page += 1
    last_date = ts_to_date(oldest_ts) if oldest_ts else "?"
    print(f"    페이지 {page:>3} | +{len(items):>3} (누적 {len(collected)}) | 최근: {last_date}")
```

Problem- each response has limits of maximum of 100 items:

- A single API call is insufficient to retrieve the entire dataset since some subreddits contain hundreds or thousands of relevant posts

Solution- Cursor-based pagination:

- Extract the created_utc of the oldest post and pass it into the before parameter of the subsequent request
- Repeat until returns are fewer than 100 items

# 3. Methodology- Crawling

## Reddit Crawling results

- Collected 95382 posts in total
- Saved as JSON for preprocessing prior to database storage

# 3. Methodology
# : NoSQL DB

# 3. Methodology- Database

- Data Preprocessing

```
unique_data = []
seen = set()

for doc in processed:
    key = doc["post_text"] + "||" + doc["comment_text"]

    if key in seen:
        continue

    seen.add(key)
    unique_data.append(doc)
```

- **1. Data Integration**
- aggregate raw data from multiple sourcesinto a single dataset

- **2. Duplicate Removal**
- Duplicate posts may occur due to repetitive content or multiple keyword matching within a single post
  - combine title and main content into a single field called post_text, and merge all comments into comment_text
  - remove the duplicates only when both post_text and comment_text are identical
  - → allows posts with different comment reactions to be preserved for analysis

# 3. Methodology- Database

- Data Preprocessing

```
KEYWORDS = [
    "ai", "artificial intelligence",
    "chatgpt", "claude","gemini","gpt",
    "llm","에이아이", "인공지능","지피티",
    "클로드", "제미나이","인공 지능",
]

def contains_ai(text):
    text = (text or "").lower()
    for keyword in KEYWORDS:
        if keyword in text:
            return True
    return False
```

- **3. Content Filtering**
- irrelevant data (e.g., words containing “ia” such as “lotteria”) were unintentionally collected during crawling
  - only data containing predefined keywords (e.g., AI, ChatGPT, Gemini, Artificial Intelligence) were retained
  - The filtering process was case-insensitive (e.g., ChatGPT, chatgpt, CHATGPT were treated equally).

# 3. Methodology- Database

- Data Preprocessing

```
def normalize_text(text):
    text = text.lower()

    # 특수문자
    text = re.sub(r"!{2,}", "!", text)
    text = re.sub(r"\?{2,}", "?", text)
    text = re.sub(r"\.{2,}", ".", text)

    # 한글
    text = re.sub(r"(ㅋ){2,}", "ㅋㅋ", text)
    text = re.sub(r"(ㅎ){2,}", "ㅎㅎ", text)

    return text.strip()
```

- **4. Special Character Normalization**
- excessive repetition of special characters may distort sentiment analysis results
  - Repeated symbols such as !, ?, . were reduced to a single instance
  - Repeated Korean expressions like ㅋ, ㅎ were normalized to fixed forms ("ㅋㅋ" and "ㅎㅎ")

# 3. Methodology- Database

- Data Preprocessing

```
def detect_language(text):
    text = text or ""

    # 한글 포함 여부 확인
    if re.search(r"[가-힣]", text):
        return "ko"
    else:
        return "eng"
```

- **5. Add Fields**
  - Language field: Labeled as “ko” if Korean characters were present, otherwise “eng”
  - Source field: Identified the community platform by detecting relevant substrings within the text

# 3. Methodology- Database

- Data Preprocessing

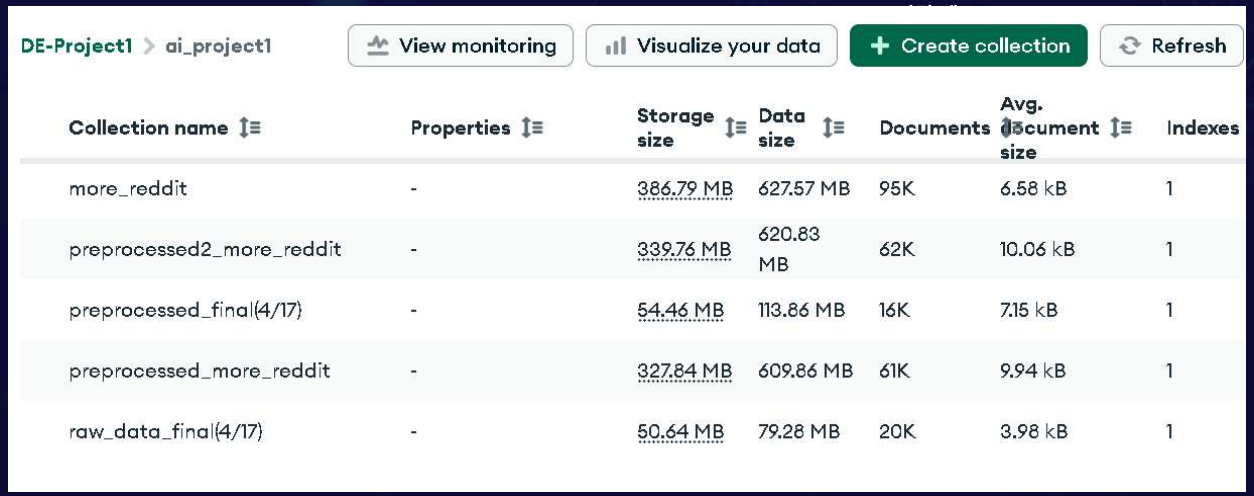
```python
def detect_source(text):
    text = str(text).lower()

    if "subreddit" in text:
        return "reddit"

    if "video_id" in text:
        return "youtube"

    if "board_name" in text and "is_pann_pick" not in text:
        return "blind"

    if "naver_news" in text:
        return "naver"

    if "dcinside" in text or "gallery_name" in text:
        return "dcinside"

    if "is_pann_pick" in text:
        return "natepann"

    if "guardian" in text:
        return "guardian"

    if "everytime" in text:
        return "everytime"
```

# 3. Methodology- Database

- Data Upload

MongoDB Atlas

- **Upload Data**
  - Uploaded the preprocessed dataset to MongoDB
  - All team members had access to and could modify the data




| Collection name | Properties | Storage size | Data size | Documents | Avg. document size | Indexes |
|---|---|---|---|---|---|---|
| more_reddit | - | 386.79 MB | 627.57 MB | 95K | 6.58 kB | 1 |
| preprocessed2_more_reddit | - | 339.76 MB | 620.83 MB | 62K | 10.06 kB | 1 |
| preprocessed_final(4/17) | - | 54.46 MB | 113.86 MB | 16K | 7.15 kB | 1 |
| preprocessed_more_reddit | - | 327.84 MB | 609.86 MB | 61K | 9.94 kB | 1 |
| raw_data_final(4/17) | - | 50.64 MB | 79.28 MB | 20K | 3.98 kB | 1 |

# 3. Methodology- Database

- Data Upload

```
MONGO_URI = "mongodb+srv://joyinjune_db_user:puda731@de-project1.gjdlngt.mongodb.net/?appName=DE-Project1"
client = MongoClient(MONGO_URI)

db = client["ai_project1"]
collection = db["preprocessed2_more_reddit"]

batch_size = 1000

for i in range(0, len(ai_data), batch_size):
    batch = ai_data[i:i+batch_size]

    try:
        collection.insert_many(batch)
        print(f"{i} ~ {i+len(batch)} 업로드 완료")
    except Exception as e:
        print("에러:", e)
```

- **Upload Data - Code**
  - Used the shared MongoDB URI and database name
  - Uploading tens of thousands of records at once caused errors, so the data was split into multiple batches for upload

# 3. Methodology- Database

- Data Upload

(Note) Error message that occurs when attempting to upload all data at once without batching

```
e 462, in _execute_batchresult = self.write_command(bwc, cmd, request_id, msg, to_send, client)  # type: ignore[arg-type
]File "C:\Users\82105\AppData\Roaming\Python\Python313\site-packages\pymongo\synchronous\helpers.py", line 47, in innerr
eturn func(*args, **kwargs)File "C:\Users\82105\AppData\Roaming\Python\Python313\site-packages\pymongo\synchronous\bulk.
py", line 274, in write_commandreply = bwc.conn.write_command(request_id, msg, bwc.codec)  # type: ignore[misc]File "C:\
Users\82105\AppData\Roaming\Python\Python313\site-packages\pymongo\synchronous\pool.py", line 486, in write_commandreply
 = self.receive_message(request_id)File "C:\Users\82105\AppData\Roaming\Python\Python313\site-packages\pymongo\synchrono
us\pool.py", line 454, in receive_messageself._raise_connection_failure(error)~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^File
"C:\Users\82105\AppData\Roaming\Python\Python313\site-packages\pymongo\synchronous\pool.py", line 628, in _raise_connect
ion_failure_raise_connection_failure(self.address, error, timeout_details=details)~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^File "C:\Users\82105\AppData\Roaming\Python\Python313\site-packages\pymongo\pool_shared
.py", line 148, in _raise_connection_failureraise AutoReconnect(msg) from errorpymongo.errors.AutoReconnect: ac-qidi8pd-
shard-00-01.gjdlngt.mongodb.net:27017: connection closed (configured timeouts: connectTimeoutMS: 20000.0ms
```

# 3. Methodology- Database

- Data Upload


```
_id: ObjectId('69e1f7a99ef77383ef7cb1ff')
post_id : "c0broyph"
keyword : "AI"
board_name : "주식·투자"
company : "이마트"
title : "Ai 거품 개헛소리 ㅋㅋㅋㅋㅋㅋ"
content : "물론 참과 거짓은 가려진다. 많은 스타트업 업체는 망하고 빅테크 일부만 남겠지. AI로 작업 효율이 좋아지면 덜 쓴다고? 한 …"
author : "익명"
created_at : "2026-04-15T00:46:38"
like_count : 2
view_count : 194
comment_count : 2
▸ comments : Array (2)
post_text : "ai 거품 개헛소리 ㅋㅋ 물론 참과 거짓은 가려진다. 많은 스타트업 업체는 망하고 빅테크 일부만 남겠지. ai로 작업 효율이 …"
comment_text : "구글 주목 ai가 대중화된지 몇년 안됐는데도 체감효과가 상당하긴해 "
language : "ko"
source : "blind"
```


- **Example of uploaded data format**
  - a single record crawled from Blind

# 3. Methodology
# : Data Analysis

# 3. Methodology-Data Analysis

- BERTopic


- **Approach (Design) :**
- Beyond simple frequency (LDA) → Contextual Embedding
- Extracts dynamic topics from unstructured community data (Everytime, Reddit)
- **Rationale (Selection Reason) :**
- N-gram Support: Captures multi-word concepts (e.g., Generative AI)
- Noise Robustness: Groups similar opinions despite slang
- Capturing contextual meaning beyond simple frequency-based models

# 3. Methodology-Sentiment Analysis

- HuggingFace multilingual

- **Approach (Design) :**
- Unified sentiment model for Korean & English datasets
- Classifies text into 5 sentiment levels
- Probability-based scoring using Softmax
- **Rationale (Selection Reason) :**
- Multilingual Capability: Supports both Korean and English
- Domain Fit: Trained on Twitter (social media) data
- Robust to informal language, slang, and emojis
- Accurate sentiment analysis on multilingual community data

# 3. Methodology-Visualization

- Plotly


- **Approach (Design) :**
- Transforms analyzed data into visual charts (distribution, trends)
- Compares topic proportions and sentiment distributions
- **Rationale (Selection Reason) :**
- Supports various chart types (bar, line, etc.)
- Provides interactive features (zoom, hover, filtering)
- Enables intuitive data exploration and comparison
- Enhancing data interpretation through interactive visualization

# 4. Implementation

# 4. What We Analyze

**1) Public Opinion Trends Over Time (Korea vs. Western)**
→ Observing shifts in public opinion across stages (Before / Early / Mid / Current)
(Naver News & The Guardian)

**2) Comparative Analysis of Community Reactions**
→ Examining differences across online communities
(Blind, DCInside, YouTube, Reddit)

**3) Occupation-Based Reaction Analysis**
→ How responses differ across professions
(Reddit Only)

# 4. Database Data Retrieval

```python
def get_data_from_db(collection_name):
    try:
        # DB 연결
        db = get_db()

        # 컬렉션 선택
        collection = db[collection_name]

        # 데이터 로드
        data_list = list(collection.find())
        df = pd.DataFrame(data_list)

        print(f"{collection_name} → {len(df)}개 데이터 로드")
        return df

    except Exception as e:
        print(f"연결 실패: {e}")
        return None
```

- A data loading pipeline that converts MongoDB data into a DataFrame for analysis.

# 4. BERTopic-Stopwords

**Stopwords Example**

```
STOP_WORDS = [
    # ========================
    # 영어 기본 불용어
    # ========================
    "is", "are", "was", "were", "be", "been", "being", "the", "a", "an",
    "on", "in", "at", "to", "for", "of", "about", "and", "or", "but",
    "this", "that", "these", "those", "it", "its", "they", "them", "their",
    "we", "you", "he", "she", "his", "her",

    # ========================
    # AI/도메인 불용어
    # ========================
    "ai", "artificial", "intelligence", "model", "data", "인공지능", "chatbots"

    # ========================
    # 한국어 조사/기능어
    # ========================
    "이", "그", "저", "것", "수", "등", "때문", "정도", "경우", "은", "는", "이", "가",
    "을", "를", "에", "에서", "으로", "로", "와", "과", "도", "만", "까지", "부터", "까지",

    # ========================
    # 한국어 동사/형용사 (자주 노이즈 되는 것)
    # ========================
    "하다", "되다", "있다", "없다", "이다", "아니다", "같다", "보다",
```

- Defined custom stopword list for noise reduction
- Removed high-frequency but low-information terms such as: **“AI”, “artificial intelligence”, “인공지능”**
- Included English and Korean functional words (e.g., particles, prepositions)

**+ a**

# 4. BERTopic-Model Configuration

```
topic_model = BERTopic(
    embedding_model=embedding_model,
    hdbscan_model=hdbscan_model,
    vectorizer_model=vectorizer_model,
    verbose=True,
    calculate_probabilities=True
)
```

- Sentence Transformer used for text embeddings
- HDBSCAN applied for clustering documents into topics
- CountVectorizer used for term frequency representation
- Combined framework enables unsupervised topic discovery

*Embedding model - paraphrase-multilingual-MiniLM-L12-v2, which is effective for multilingual semantic similarity tasks.

# 4. Hugging Face-Model Setting

```
sentiment_model = pipeline(
    "sentiment-analysis",
    model="nlptown/bert-base-multilingual-uncased-sentiment",
    tokenizer="nlptown/bert-base-multilingual-uncased-sentiment",
    device=0  # GPU 없으면 -1
)
```

- Utilize the pretrained model **"nlptown/bert-base-multilingual-uncased-sentiment"** for multilingual sentiment classification.

# 4. Hugging Face-Model Setting

```python
def map_sentiment(label):
    num = int(label.split()[0])

    if num <= 2:
        return "negative"
    elif num == 3:
        return "neutral"
    else:
        return "positive"
```

5 stars

Positive / Negetive / Neutral

# 4. Hugging Face-Model Execution

**<Fuction Definition>**

```python
def analyze_batch(texts):
    texts = [clean_text(t)[:512] for t in texts]

    results = sentiment_model(
        texts,
        batch_size=32,
        truncation=True
    )

    return [map_sentiment(r["label"]) for r in results]
```

**<Execution>**

```python
batch_size = 32
texts = df_filtered["post_text"].tolist()

sentiments = []

for i in tqdm(range(0, len(texts), batch_size)):
    batch = texts[i:i+batch_size]
    sentiments.extend(analyze_batch(batch))

df_filtered["sentiment"] = sentiments
```

Given the time-consuming nature of sentiment analysis, **batch** processing was utilized to enhance computational efficiency.

# 4. Setup per Analysis: Public Opinion Trends Over Time (Korean vs. Western Communities)

```python
def time_group(date):
    if date <= pd.Timestamp("2022-11-29"):
        return "before"
    elif date <= pd.Timestamp("2023-12-29"):
        return "early"
    elif date <= pd.Timestamp("2025-01-28"):
        return "mid"
    else:
        return "current"

df_filtered["time_group"] = df_filtered["created_at"].apply(time_group)
```

← **Time-Based Segmentation**
(before / early / mid / current)

```python
def map_region(source):
    if source == "naver":
        return "korea"
    elif source == "guardian":
        return "western"
    else:
        return "other"

df_filtered["region"] = df_filtered["source"].apply(map_region)

df_filtered = df_filtered[
    (df_filtered["region"] == "korea") & (df_filtered["language"].astype(str).str.startswith("ko"))
    |
    (df_filtered["region"] == "western") & (df_filtered["language"].astype(str).str.startswith("en"))
].copy()
```

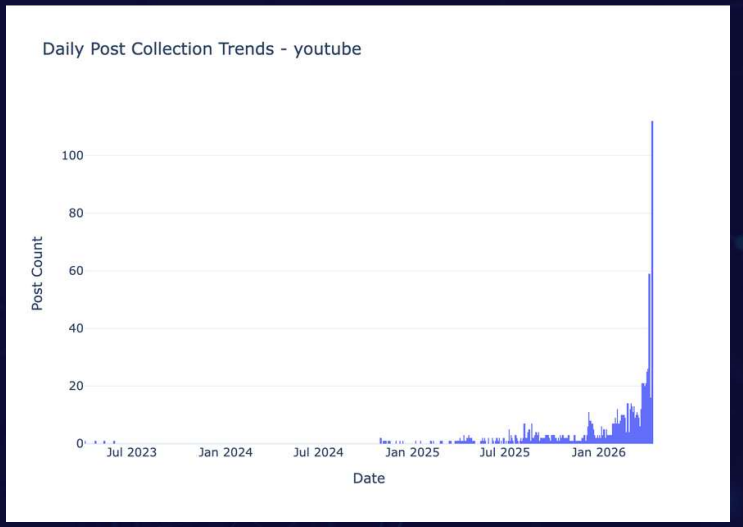
← **Language-Based Filtering**
(korea / western)

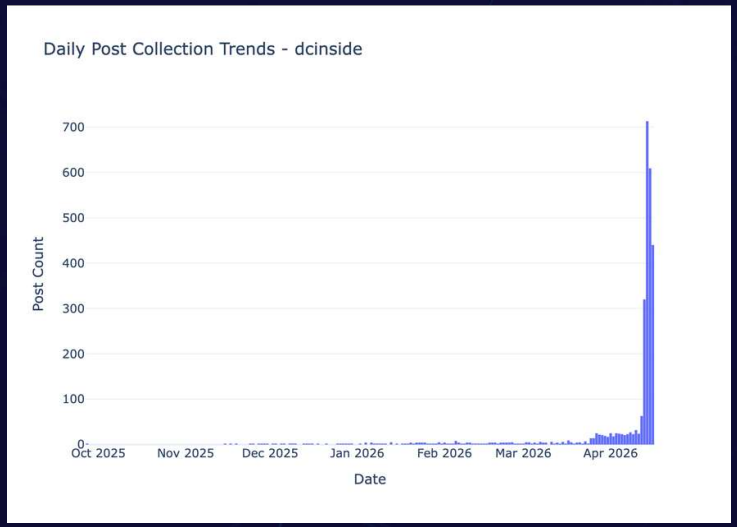
# 4. EDA: Distribution of Data Collection Periods

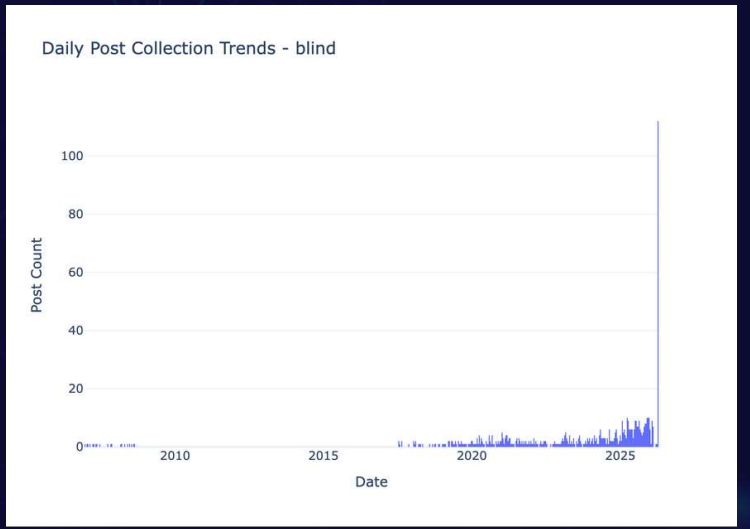
- Collected exactly the latest 300 posts from each of the 8 communities to ensure an unbiased, equal-volume baseline.
- Natural Timeline Variance: The time span required to gather 300 posts varies by platform.
- This reflects the unique characteristics and discussion velocity of each community.
- Fairness in Approach: Rather than arbitrarily cutting off by a specific date, volume-based extraction ensures we capture the most relevant and recent discussions across all sources equally.

YouTube: Jul 2023 ~ Apr 2026 (approx. 3 years)
DC Inside: Nov 2025 ~ Apr 2026 (approx. 5 months)
Reddit: Jul 2022 ~ Apr 2026 (approx. 3.5 years)
Blind: 2008 ~ Apr 2026 (approx. 18 years)


Daily Post Collection Trends - reddit
Post Count
300
250
200
150
100
50
0
Jul 2022
Jan 2023
Jul 2023
Jan 2024
Jul 2024
Jan 2025
Jul 2025
Jan 2026
Date


Daily Post Collection Trends - youtube
Post Count
100
80
60
40
20
0
Jul 2023
Jan 2024
Jul 2024
Jan 2025
Jul 2025
Jan 2026
Date


Daily Post Collection Trends - dcinside
Post Count
700
600
500
400
300
200
100
0
Oct 2025
Nov 2025
Dec 2025
Jan 2026
Feb 2026
Mar 2026
Apr 2026
Date


Daily Post Collection Trends - blind
Post Count
100
80
60
40
20
0
2010
2015
2020
2025
Date

# 4. BERTopic-Community Analysis Pipeline

**Community-specific Topic Modeling Process**

```
df_filtered = df[df["source"] == target_source]

df_filtered["content"] = (
    df_filtered["title"].fillna("") + " " +
    df_filtered["content"].fillna("")
)

documents = df_filtered["content"].tolist()
```

- Retrieved data from MongoDB collection
- Separated data by community using source
- Merged title and content into unified text
- Filtered short and invalid documents
- Prepared clean document set for each community

# 4. BERTopic-Topic Modeling Execution

```
# 2. Topic summaries generated per community
info = topic_model.get_topic_info()

# 3. Results saved for comparison across platforms
info.to_csv(f"topic_info_{target_source}.csv", index=False, encoding="utf-8-sig")
```

- Selects a specific community dataset
- Constructs unified text input
- Applies topic modeling to extract discussion clusters
- **Output**
- Topic summaries generated per community
- Results saved for comparison across platforms

# 4. Setup per Analysis: Occupation-Based Reaction Analysis with MMR

**Problem: Highly similar terms within each job group**
(e.g., Natural Science - chemistry, chemical, chemists, organic chemistry)

**MMR**
(Maximal Marginal Relevance)

**"A method that reduces similar words and selects only diverse, representative keywords."**

**0 ≤ diversity ≤ 1**

**Selecting only similar words.**

**Maximizing diversity**

# 4. Plotly- Data Structuring

```
time_order = ['before', 'early', 'mid', 'current']
df['time_group'] = pd.Categorical(df['time_group'], categories=time_order, ordered=True)
```

- Defines a chronological sequence: Before → Early → Mid → Current
- Ensures the X-axis follows a logical timeline rather than alphabetical order.

# 4. Plotly-Visual Encoding

```
fig = px.bar(
    df,
    x='time_group',
    y='ratio',
    color='sentiment',
    facet_col='region',  # 한국과 영미권을 옆으로 나란히 비교
    barmode='group',     # 막대를 옆으로 나열
    category_orders={"time_group": time_order, "sentiment": ["negative", "neutral", "positive"]},
    color_discrete_map=color_map,
    title='Comparative Analysis of Sentiment Shifts toward AI: Korea vs. Western',
    labels={'ratio': 'Ratio (%)', 'time_group': 'Analysis Period', 'sentiment': 'sentiment'},
    text='ratio',        # 막대 위에 비율 표시
    template='plotly_white'
)
```

- Color Mapping: Intuitive color coding for sentiments (Red for Negative, Green/Blue for Positive).
- Comparative Layout: Side-by-side comparison between Korea and Western regions using facet grids.

# 4. Plotly-Display Optimization

```
fig.update_traces(texttemplate='%{text:.1f}%', textposition='outside')
fig.update_layout(
    yaxis_range=[0, 100], # 비율이므로 0~100 고정
    font=dict(size=12),
    legend_title_text='Classification of emotions'
)
```

- Data Labels: Direct display of percentage values above bars for immediate readability.
- Scale Normalization: Fixed Y-axis range (0-100%) to provide an accurate ratio comparison.

# 4. Visualization

# 4. Plotly- Temporal Logic & Data Merging

```
time_order = ['before', 'early', 'mid', 'current']
df['time_group'] = pd.Categorical(df['time_group'], categories=time_order, ordered=True)

df['all_topics'] = df['top1'] + "<br>" + df['top2'] + "<br>" + df['top3']
```

- Chronological Alignment: Organizes data into a logical timeline (Before → Early → Mid → Current) to prevent alphabetical sorting.
- Text Concatenation: Merges multiple topic columns (top1, top2, top3) into a single text block for visualization.

# 4. Plotly- Scatter-based Timeline Mapping

```
fig = px.scatter(
    df,
    x='time_group',
    y='region',
    text='all_topics',
    color='region',
    title='Comparing AI Trends: Korea vs. Western Markets (Over Time)',
    labels={'time_group': 'phase', 'region': 'region'},
    template='plotly_white',
    height=500
)
```

- Coordinate Mapping: Uses the X-axis for time flow and the Y-axis for regional comparison.
- Semantic Labeling: Displays the merged topic text directly on the plot to show "what happened when."

# 4. Plotly- Annotation & Aesthetic Design

```
fig.update_traces(
    textposition='middle center',
    mode='text+markers',
    marker=dict(size=40, opacity=0.1) # 텍스트 배경에 은은한 원형 표시
)

fig.update_layout(
    font=dict(size=12),
    xaxis_title="Time Flow (Before → Current)",
    yaxis_title="Region",
    showlegend=False
)
```

- Visual Highlighting: Adds semi-transparent circular markers behind the text to serve as "soft backgrounds," improving text legibility.
- Layout Refinement: Removes redundant legends and clarifies axis titles to focus entirely on the temporal trend.

# 4. Visualization

# 4. Visualization Interpretation

1. Korea: From Industrial Opportunity to Geopolitical Risk
   - Early Phase: Focused on AI as a National Growth Engine (Keywords: Naver, Industry 4.0) and Global Copyright issues.
   - Current Phase: Shifted toward Geopolitical Dynamics (Keywords: DeepSeek, Trump, China). AI is now perceived through the  international relations and global policy shifts.
2. Western: Focus on Social Safety Nets, Ethics, and Labor
   - Early Phase: Centered on Governance and Regulation (Keywords: Safety Summit, Security, Policy) to control technological risks.
   - Current Phase: Deepened focus on Social Impact (Keywords: Jobs, Workers, Ethics). Concerns regarding labor displacement and generative AI bias (Gender/Images) have become mainstream.
3. Key Divergence in Perspectives
   - Korea (Strategic & Macro): Prioritizes Global Competitiveness. Concerns center on how external factors (U.S. politics, China's rise) affect the national AI industry.
   - Western (Value-driven & Micro): Prioritizes Human-Centric Values. Focuses on individual livelihoods, job security, and ethical integrity.

# 4. Plotly- Data Cleaning & Top-K Filtering

```
plot_df = df[df['Topic'] != -1].head(10)
```

- Noise Removal: Excludes outlier data (Topic -1) to focus on meaningful clusters.
- Top 10 Selection: Filters the most frequent topics to ensure a clean and concise visualization.

# 4. Plotly- Horizontal Semantic Mapping

```
fig = px.bar(
    plot_df,
    x='Count',
    y='Name',
    orientation='h',  # 가로 막대 그래프
    title=f'[{community_name}] Top 10 Key Reactions to AI',
    labels={'Count': 'Number of Posts', 'Name': 'Extracted Topics (Representative Keywords)'},
    color='Count',    # 빈도수에 따라 색상 변경
    color_continuous_scale='Blues', # 색상 테마 (Blues, Reds, Viridis 등)
    template='plotly_white'
)
```

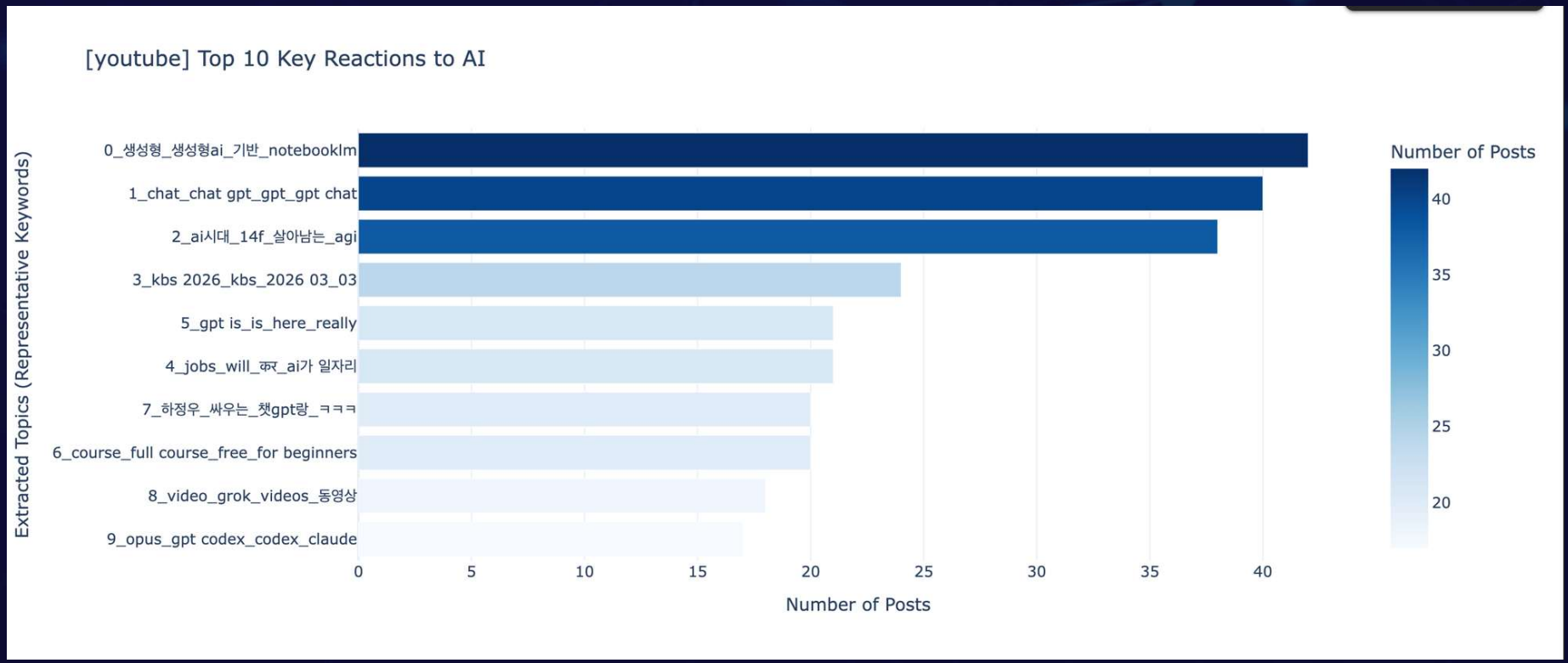
- Horizontal Layout: Enhances readability for long topic names (keywords) by using a horizontal bar format.
- Intensity Encoding: Uses a monochromatic color scale (Blues) to intuitively represent the volume of posts for each topic.

# 4. Plotly- Layout Optimization for Text

```
fig.update_layout(
    yaxis={'categoryorder': 'total ascending'}, # 빈도가 높은 게 위로 오도록 정렬
    font=dict(size=12),
    margin=dict(l=200) # 토픽 이름이 길 경우 왼쪽 여백 확보
)
```

- Frequency Sorting: Automatically sorts bars by count, ensuring the most dominant topics are positioned at the top.
- Label Accommodation: Adjusts the left margin (L=200) to prevent long representative keywords from being cut off.

# 4. Visualization



# 4. Visualization Interpretation

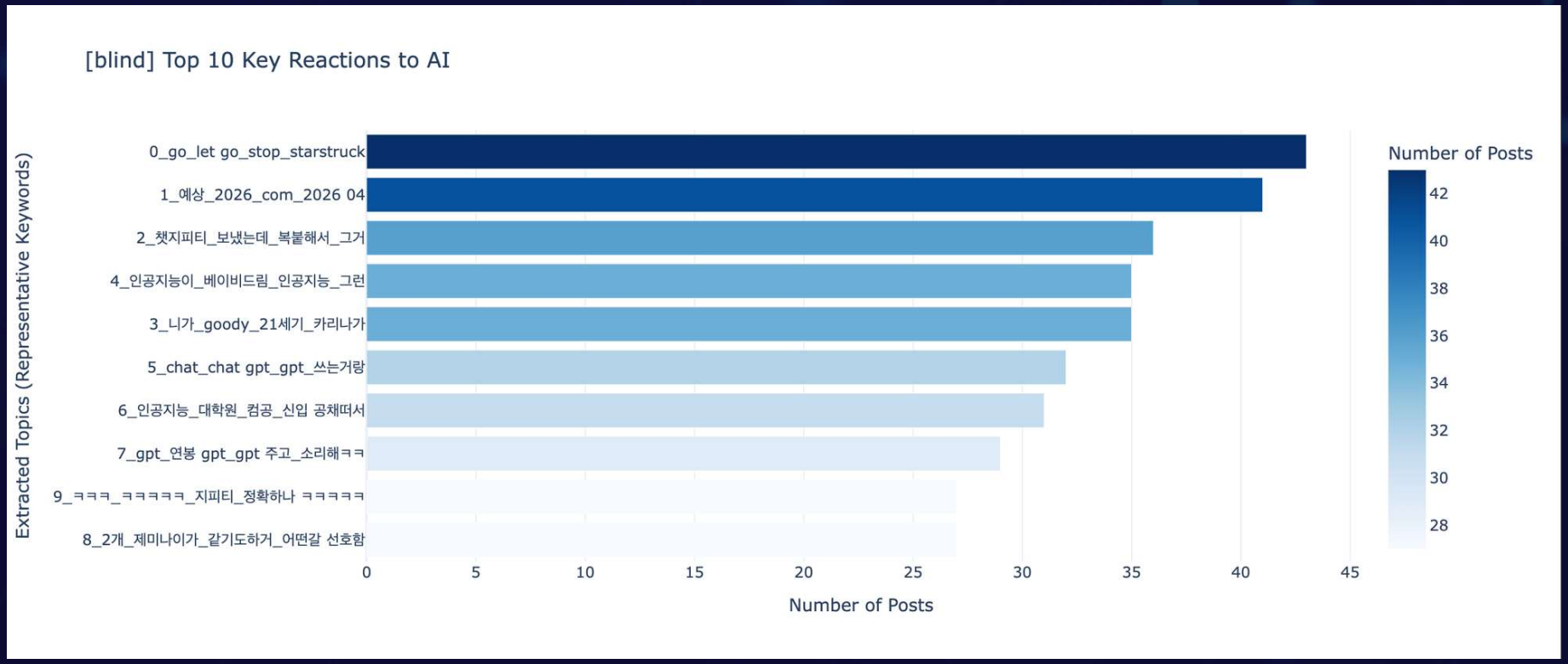
opus_gpt_codex_claude: Focused on Specialized Tool Tutorials & Reviews

kbs_2026_03_03: Mainstream news and documentaries transitioning to digital-first consumption.

course_full_course_free_for_beginners: Leveraging free digital resources to effectively bridge the technical skill gap.

Key Insight: The public is simultaneously embracing AI to enhance productivity while fearing its potential to replace human roles.

# 4. Visualization



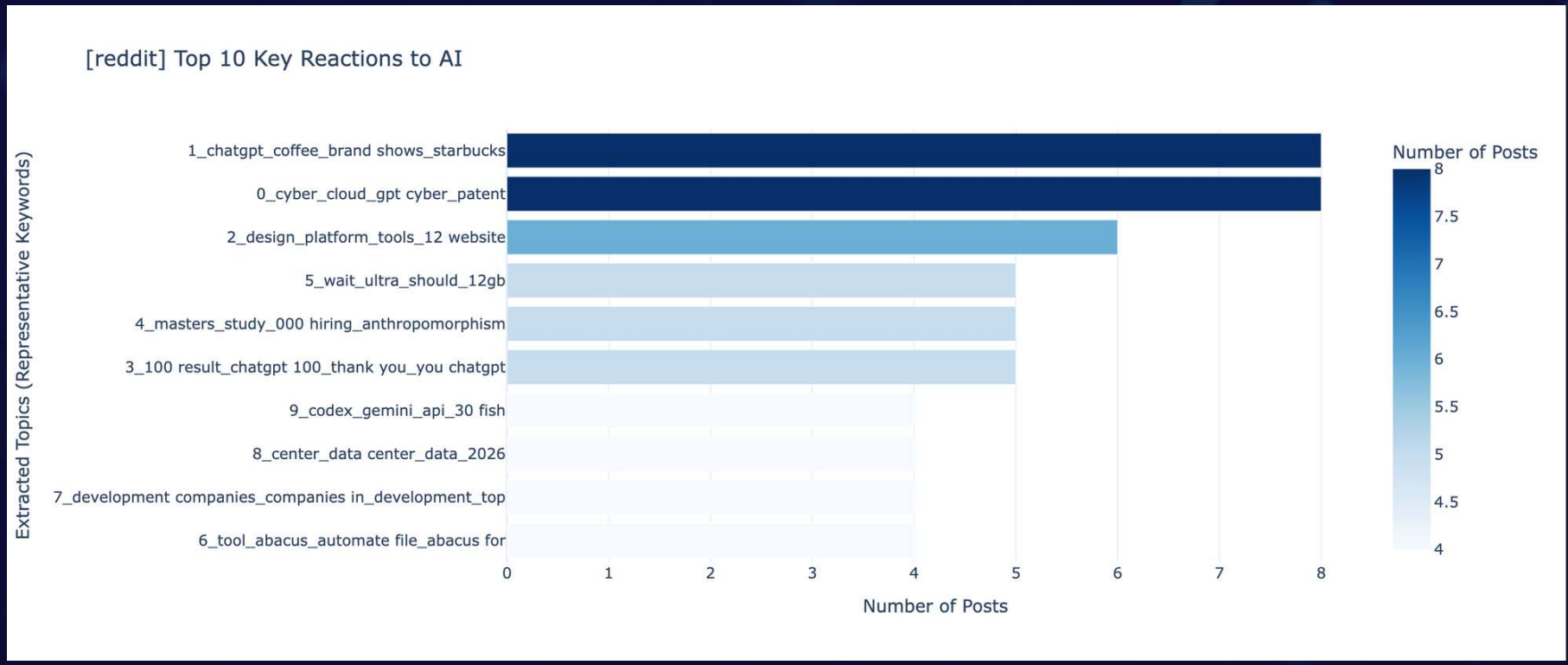
# 4. Visualization

인공지능_대학원_컴공_신입_공채떠서: Professionals are closely monitoring how AI disrupts traditional hiring standards.

gpt_연봉_gpt_주고_소리해: Intense debate over whether AI will lead to massive cost-cutting or a premium salary hike for AI experts.

Key Insight: Both Korean (Blind) and Western (Reddit) professionals share deep concerns regarding AI's impact on the economically active population. While Reddit debates systemic ethics, Blind focuses intensely on individual survival strategies, such as salary and job security.

# 4. Visualization



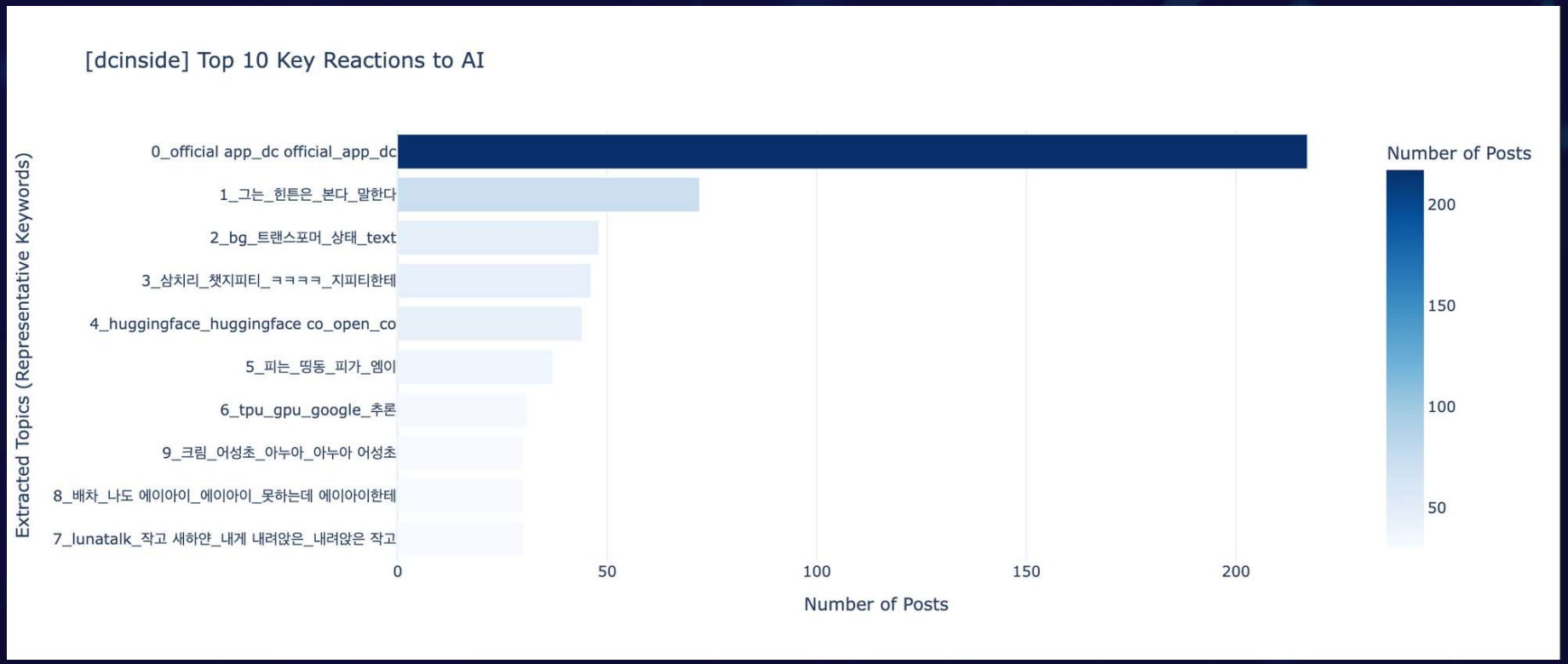
# 4. Visualization

design_platform_tools_12_website: Optimizing AI as a Professional Productivity Tool for Design and Development

cyber_cloud_gpt_cyber_patent: Debating AI Patents, Security

Key Insight: Perceived as a real-world force affecting professional expansion, and recruitment paradigms. Stronger tendency to analyze and prepare for long-term socio-economic transformations driven by AI.

# 4. Visualization



# 4. Visualization Interpretation

그는_힌튼은_본다_말한다: tatements and actions by Geoffrey Hinton, the "Godfather of AI," serve as a primary catalyst for community debate.

huggingface_open_co, tpu_gpu_google_추론: High Interest in AI Reasoning, Hardware Performance, and Open-Source Ecosystems

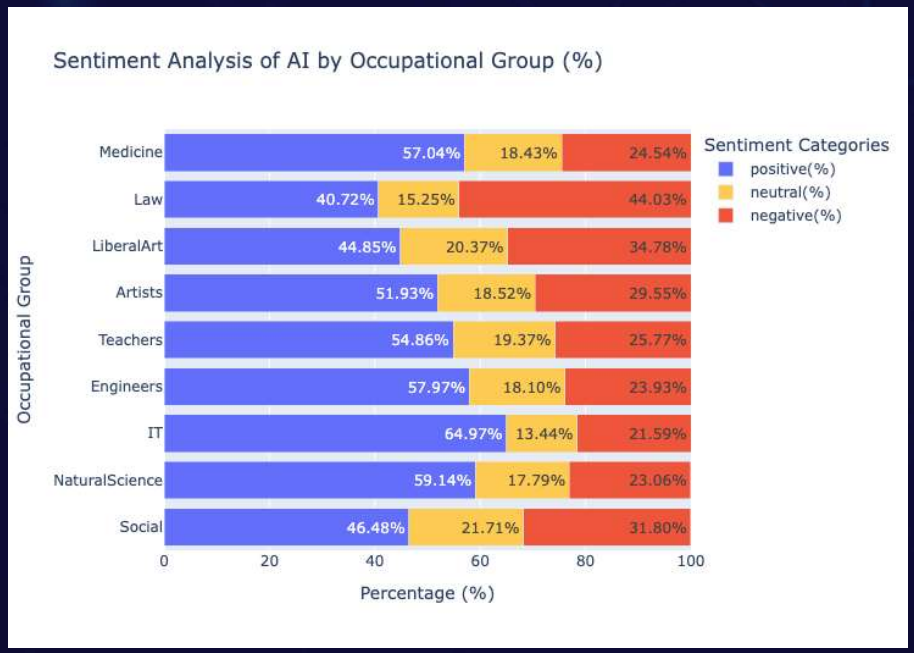
Key Insight : The Dual Structure of AI Discourse - Balancing Technical Curiosity with Playful Engagement

# 5. Result

# 5. Result-Sentiment Analysis

```python
df_melted = df.melt(
    id_vars=['group', 'total_count'],
    value_vars=['positive(%)', 'neutral(%)', 'negative(%)'],
    var_name='sentiment',
    value_name='percentage'
)
fig = px.bar(
    df_melted,
    x='percentage',
    y='group',
    color='sentiment',
    orientation='h',  # 가로 막대 그래프
    title='Sentiment Analysis of AI by Occupational Group (%)',
    labels={'percentage': 'Percentage (%)', 'group': 'Occupational Group', 'sentiment': 'Sentiment'},
    color_discrete_map={
        'positive(%)': '#636EFA',  # 파랑
        'neutral(%)': '#FECB52',   # 노랑
        'negative(%)': '#EF553B'   # 빨강
    },
    text='percentage'  # 막대 위에 수치 표시
)

# 4. 레이아웃 세부 설정
fig.update_layout(
    barmode='stack',
    yaxis={'categoryorder': 'total ascending'}, # 비율 합계 기준 정렬
    xaxis_range=[0, 100],
    legend_title_text='Sentiment Categories'
)

fig.update_traces(texttemplate='%{text:.2f}%', textposition='inside')

fig.show()
```

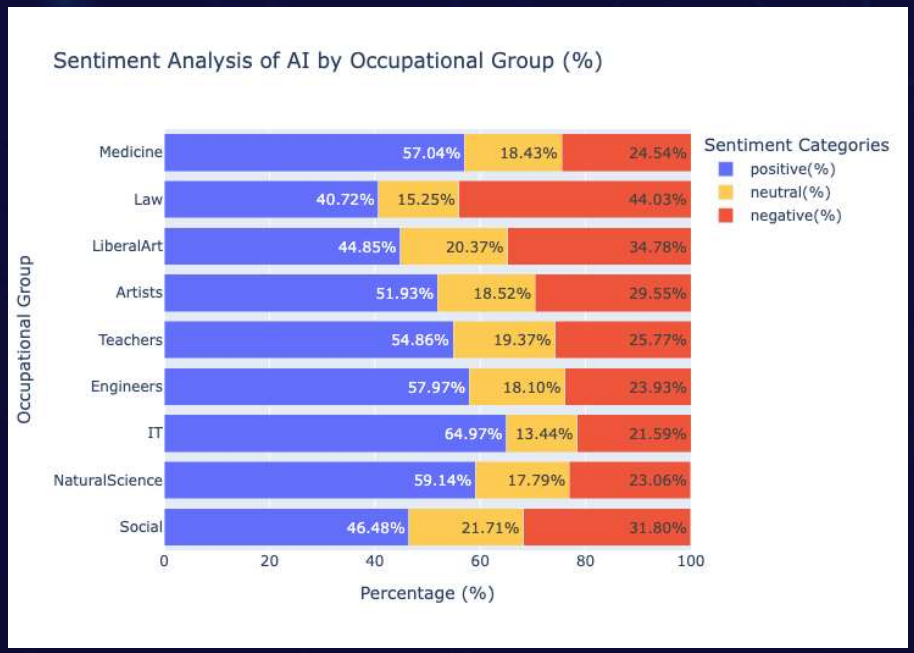
# 5. Result-Sentiment Analysis




1. IT & Tech: Strong Optimism
Highest Positivity: IT sector leads with 65.0% (lowest negative rate).
Engineers (58.0%) & Natural Sciences (59.1%) also show above-average positivity.
Insight: Higher tech familiarity translates to viewing AI as a useful tool rather than a threat.

2. Medicine & Education
Majority positive: Medicine (57.0%) & Teachers (54.9%).
Insight: These sectors highly value AI as an assistive tool to enhance work efficiency and productivity.

# 5. Result-Sentiment Analysis




3. Law: High Caution & Skepticism
Highest Negativity: Records 44.0% negative responses (Lowest positive at 40.7%).
Insight: Highly conservative stance, likely due to critical concerns over AI's reliability, ethical accountability, and potential job replacement.

4. Humanities & Social Sciences: Cautious & Neutral
Positivity remains below 50% (Social: 46.5%, Liberal Arts: 44.9%).
Highest Neutral stance (21.7% in Social Sciences).
Insight: Maintaining a reserved attitude to carefully evaluate the multifaceted societal impacts of AI.

# 5. Result-Sentiment Analysis

- Tech-Centric Fields Embrace AI: IT (65% positive), Engineering, and Natural Sciences view AI as a highly useful tool.
- Medicine & Education Focus on Utility: Healthcare (57%) and Teachers (55%) show practical optimism, valuing AI for efficiency.
- Humanities Take a Cautious Stance: Social Sciences and Liberal Arts show below 50% positivity, with high neutral rates reflecting ongoing evaluation of AI's societal impact.
- Legal Sector Remains the Most Skeptical: Law records the highest negative response (44%), prioritizing ethical responsibility and reliability concerns.

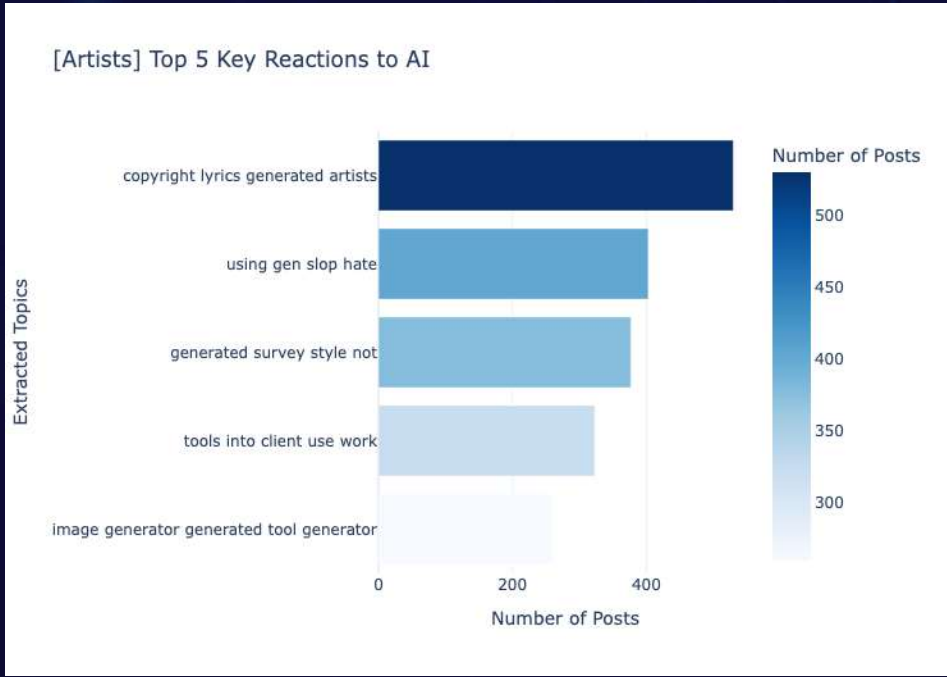
**Result : Tech-oriented professionals warmly welcome AI, whereas legal and humanities sectors maintain a highly cautious and critical perspective.**

# 5. Result-Sentiment Analysis

```python
# 1. 설정
file_path = "group_Artists_top5_topics.csv"
group_name = "Artists"

# 2. 데이터 불러오기
df = pd.read_csv(file_path)

# 3. 데이터 전처리

plot_df = df[df['topic_id'] != -1].head(5)

# 4. 시각화 구현
fig = px.bar(
    plot_df,
    x='count',
    y='name',
    orientation='h',
    title=f'[{group_name}] Top 5 Key Reactions to AI',
    labels={'count': 'Number of Posts', 'name': 'Extracted Topics'},
    color='count',
    color_continuous_scale='Blues',
    template='plotly_white'
)

# 5. 그래프 레이아웃 세부 설정
fig.update_layout(
    yaxis={'categoryorder': 'total ascending'},
    font=dict(size=12),
    margin=dict(l=200)
)

# 6. 출력
fig.show()
```

# 5. Result-Topic Modeling

**Artists**




1. Copyright & IP Protection (Top Priority)
Strong demand for legal safeguards against unauthorized AI training.

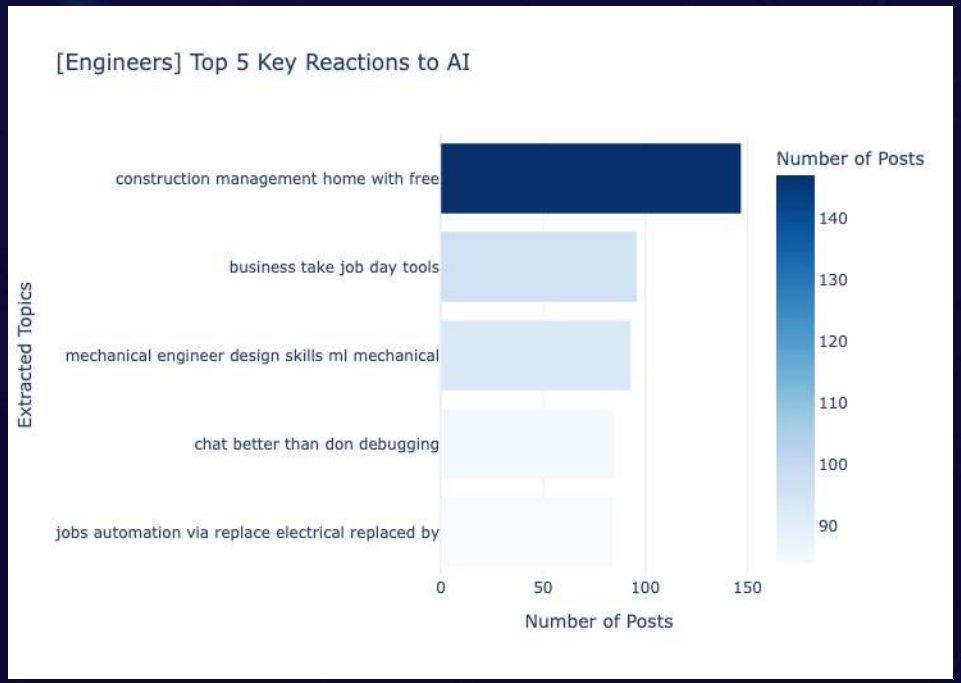
2. Aversion to AI "Slop"
Deep resentment toward mass-produced, low-quality AI content.

3. Workflow Integration
Active debate on using AI as a supplementary tool for client projects.

4. Style Mimicry & Identity
High concern over AI cloning unique artistic styles and diluting human originality.

# 5. Result-Topic Modeling

**Engineers**




1. Field Management & Efficiency
High interest in applying AI as a practical tool for construction management.

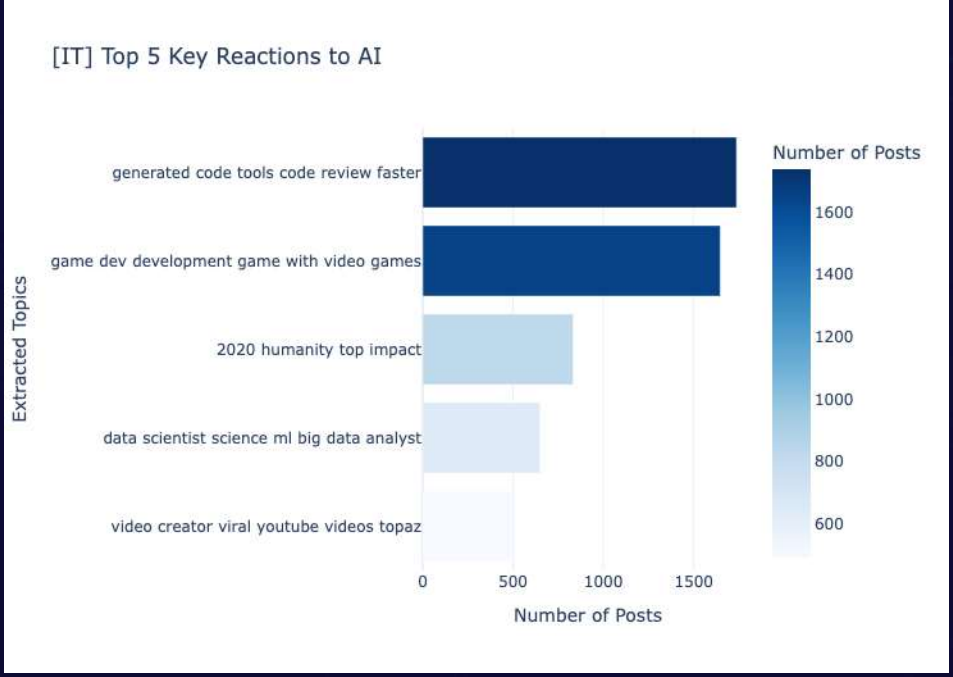
2. ML & Design Integration
Actively combining Machine Learning with traditional mechanical design skills.

3. Chatbots for Debugging
AI exceeds expectations ("better than...") for tedious debugging tasks.

4. Job Security & Automation
Real fears regarding AI "replacing" human jobs in specific sectors (e.g., electrical).

# 5. Result-Topic Modeling

**IT**




1.Unmatched Productivity (Faster Coding)
Overwhelmingly viewed as a tool to make code generation and review faster (Top topic: 1,700+ posts).

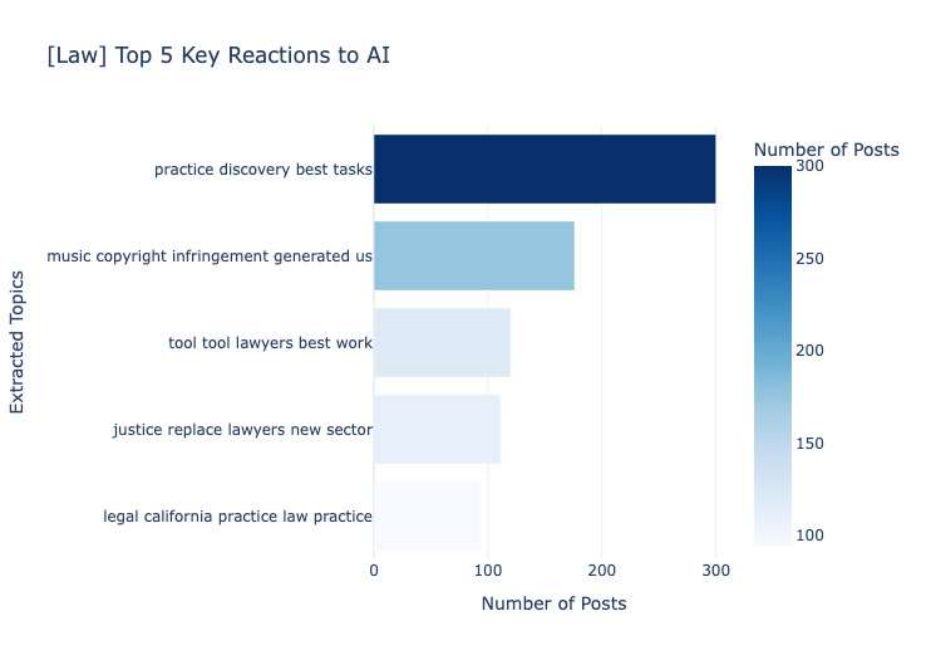
2. Game Dev & Multimedia
Driving massive shifts in "game development" and "video creation" (e.g., generating assets).

3. Advancing Data Science
Elevating analytical capabilities in "big data" and ML models.

4. Macro Impact
Discussions extending to AI's broader impact on humanity.

# 5. Result-Topic Modeling

**Law**




1. E-Discovery Efficiency
Top interest lies in using AI to process massive discovery tasks effectively.

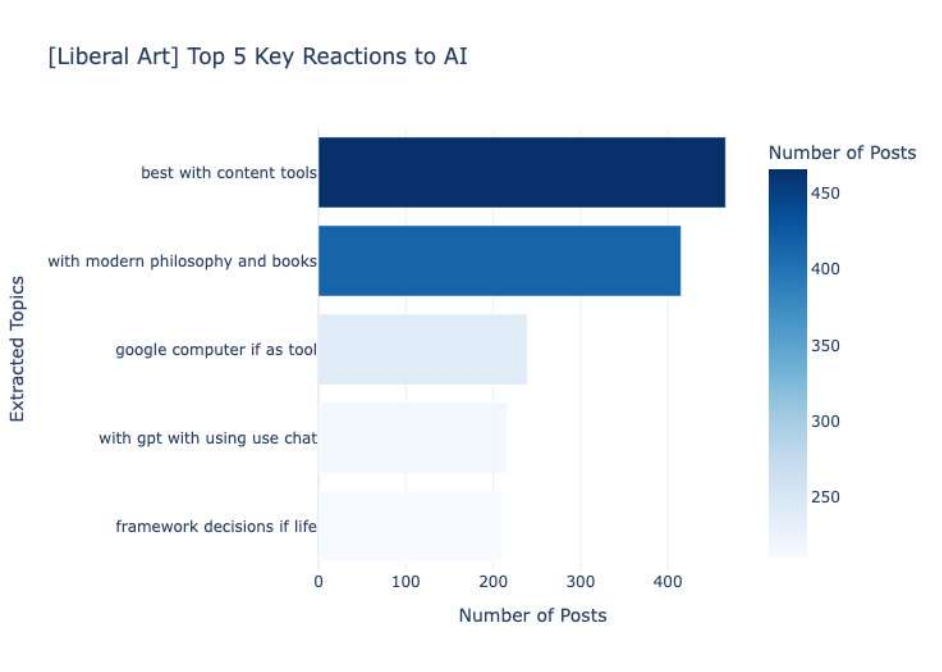
2. Copyright Infringement
Analyzing AI strictly from a legal and precedent-setting perspective (focusing on infringement).

3. Justice vs. Replacing Lawyers
Philosophical and professional alarms about AI replacing lawyers and its effect on justice.

4. Frameworks & Regional Practices
Focusing on turning AI into the best tool while monitoring fast-moving regional regulations (e.g., California).

# 5. Result-Topic Modeling

**Liberal Art**




1. Content Creation Utility
Highly valued as the best tool for writing, planning, and content creation.

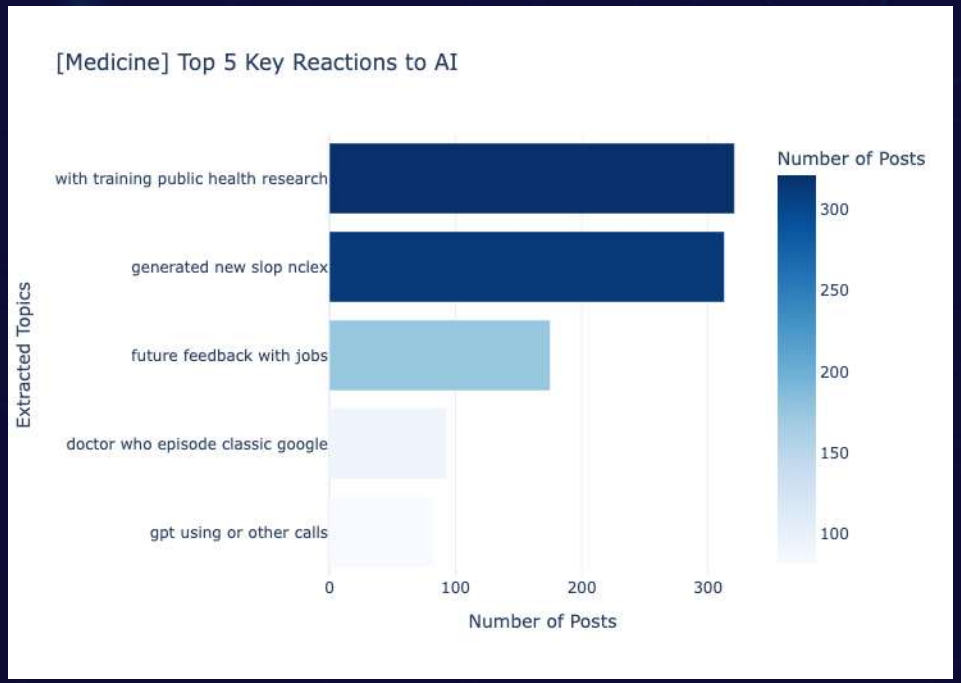
2. Philosophical Reflection
Analyzing AI through modern philosophy and the wisdom of classical books.

3. The "Mere Tool" Debate
Debating whether AI is just a functional tool like Google or an entity that exceeds human capabilities.

4. Ethics & Life Decisions
Deep concerns about AI influencing human decisions and the urgent need for a moral framework.

# 5. Result-Topic Modeling

**Medicine**



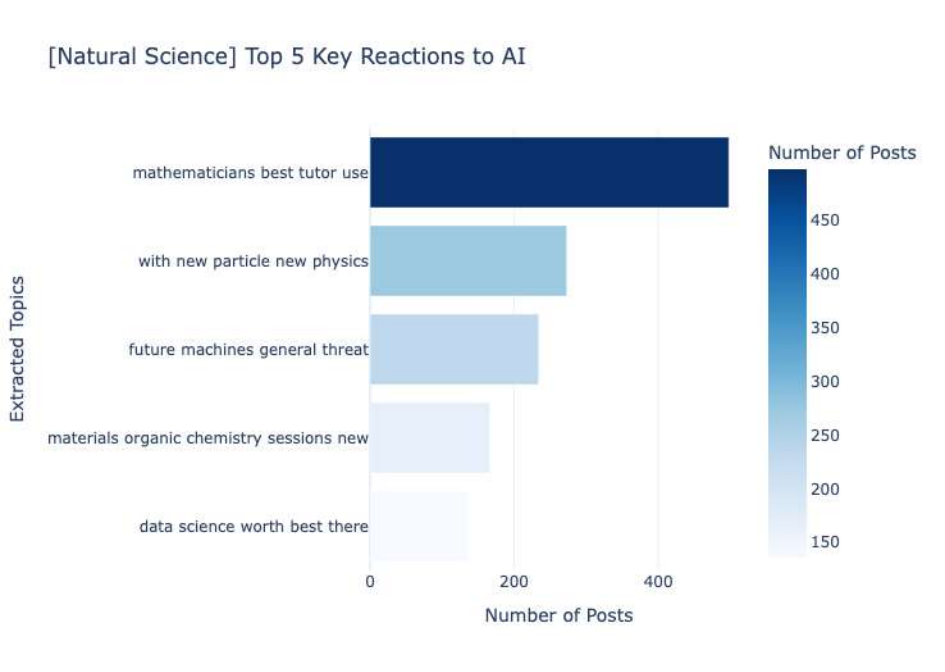

1. Public Health & Research
Top Focus: Moving beyond individual care to improve broad public health systems and medical research.

2. Medical Exams & Quality Control (NCLEX)
High interest in using AI for medical exams like the NCLEX. However, the presence of the word slop indicates a strong need to verify the quality and accuracy of AI-generated educational materials.

3. Future Workflows & Admin
Actively exploring AI ("GPT") for administrative tasks (calls) and real-time patient feedback.

# 5. Result-Topic Modeling

**Natural Science**




1. The Best Tutor for Complex Science
Overwhelmingly embraced as an intelligent personal tutor for mathematicians and scientists (Top topic).

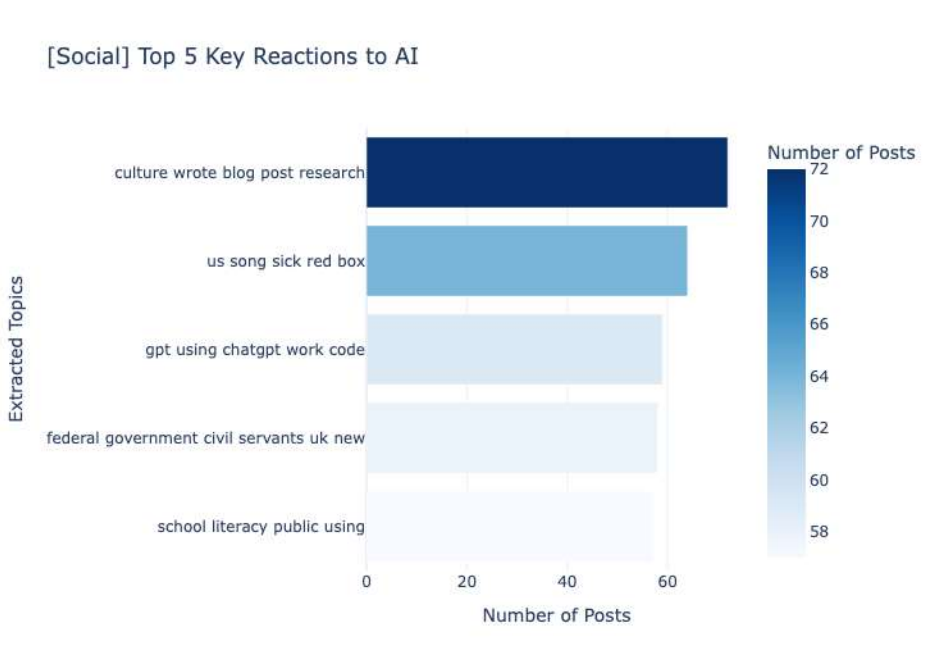
2. Accelerating Breakthroughs
Driving massive innovations in discovering new particles, advancing new physics, and simulating organic chemistry materials.

3. Data-Driven Paradigm Shift
AI has become an essential "data science" capability to prove the "worth" of scientific discoveries.

4. The AGI Threat
Maintaining a critical, existential caution regarding the "general threat" of future intelligent machines.

# 5. Result-Topic Modeling

**Social**



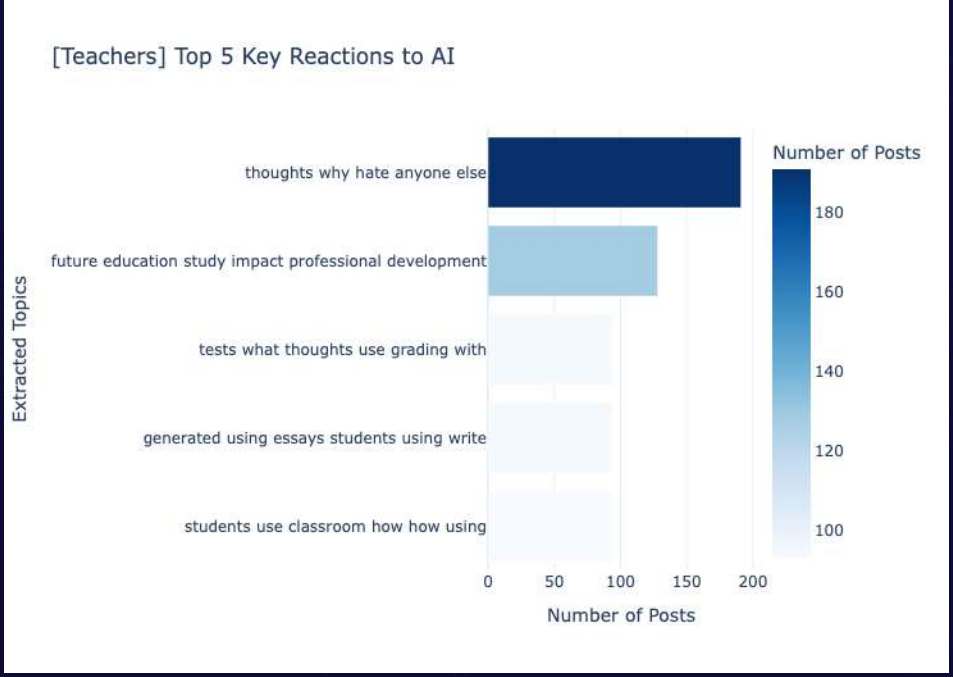

1. Public Sector & Governance
Deeply analyzing how the federal government and civil servants adapt to AI, focusing on macro-policy shifts.

2. Digital Literacy & Educational Divide
Strong focus on public literacy and whether schools are equipped to prevent tech-driven social inequality.

3. Cultural Shifts & Practical Use
Utilizing ChatGPT for practical work while observing how AI alters pop culture (e.g., songs) and communication.

# 5. Result-Topic Modeling

**Teachers**




1. Emotional Fatigue & Skepticism
The #1 topic (thoughts why hate) reflects widespread burnout, confusion, and emotional resistance to the disruption of traditional education.

2. The Essay Crisis & Plagiarism
Immense stress and sensitivity over students using AI to write essays, viewing it as a direct threat to academic integrity.

3. AI for Grading vs. Ethics
Practical debates on whether to use AI for grading tests—balancing workload efficiency with fairness.

4. Shaping Classroom Guidelines
Urgent discussions on how to properly guide students in the classroom and adapt their own professional development.

# 5. Result-Sentiment Analysis

**Artists -** AI is largely viewed as an existential threat to creativity and IP, sparking urgent calls for institutional regulations.

**Engineers -** Engineers welcome AI as a powerful optimization and debugging partner, yet strictly monitor the looming threat of job automation.

I**T-** IT experts do not see AI as a novelty; it is heavily integrated as an essential accelerator for coding, data, and digital creation.

**Law-** AI is a powerful tool for legal efficiency, yet a profound threat to legal foundations and human judgment.

**Liberal Art-** Humanities embrace AI for productivity but critically examine its threat to human agency, focusing on establishing strict ethical frameworks.

# 5. Result-Sentiment Analysis

**Medicine -** Medical professionals view AI as a powerful partner for systemic health improvements, but strictly emphasize quality control for AI-generated medical education.

**Natural Science -** Scientists aggressively leverage AI to accelerate unprecedented discoveries, while simultaneously warning of the existential risks of future Artificial General Intelligence.

**Social-** Social scientists look beyond personal utility, critically analyzing AI's macro-impact on government policies, social inequality, and public digital literacy.

**Teachers -** Educators face a dual crisis: battling severe emotional burnout and student misuse (cheating), while urgently trying to establish new classroom norms.

# 팀 구성

## 이시은 2466044

컴퓨터공학과
PM, NoSQL DB

## 장윤서 2492034

데이터사이언스학과
NoSQL DB

## 백지현 2492012

데이터사이언스학과
Crawling

## 정지영 2392032

데이터사이언스학과
Crawling

## 전서영 2392047

데이터사이언스학과
Data Visualization
& Analysis

## 유선하 2414022

데이터사이언스학과
Data Visualization
& Analysis

## 김채은 2492045

데이터사이언스학과
Data Visualization
& Analysis

# Collaboration & Workflows

- **Workflows** Crawling → DB → Analysis → Visualization

- **Tools**



Version Control &
Configuration Management


Documentation &
Meeting Notes


Weekly Meetings
(Tue, 22:30 via Google Meet)

Repository: https://github.com/DE-Team5/ai-sentiment-analysis

# Thank You
# For Watching

End Slide

# References


AI Diffusion Report: A Widening Digital Divide

<Data Crawling>

https://makes-sense.tistory.com/9

https://velog.io/@psr2110/%EC%9E%90%EC%97%B0%EC%96%B4%EC%B2%98%EB%A6%AC-%ED%94%84%EB%A1%9C%EC%A0%9D%ED%8A%B81%EC%A3%BC%EC%B0%A8-%ED%81%AC%EB%A1%A4%EB%A7%81

https://wdprogrammer.tistory.com/39

https://kaya-dev.tistory.com/32

https://github.com/HackerNews/API

<Text Data Analysis & Visualization>

nlptown/bert-base-multilingual-uncased-sentiment · Hugging Face

https://maartengr.github.io/BERTopic/index.html